Formal Semantics and Theorem Reference for Mergeable Replicated Data Types

Anonymous Authors

Abstract

This document gives a bottom-up reference for the public formal theory of mergeable replicated data types (MRDTs) mechanized in Lean 4. It starts with events and datatype signatures, defines the version-DAG operational semantics, then explains how finite event histories determine stored states, which merge property preserves that explanation, and how operation-generation constraints connect it to an independent client specification. It also covers merge-base construction for version DAGs without a single greatest common ancestor and two independent garbage-collection refinements. Definitions are ordered by semantic dependence rather than Lean file order. Every principal definition and theorem has its exact Lean declaration and a source link. Proof-local lemmas are omitted from the main line; the historical binary development is isolated in an appendix.

Contents

1 Purpose and evidence boundary **2**

2 Primitive types and datatype signatures **3**
2.1 Identifiers and events . . . 3
2.2 Finite replay . . . 4

3 Version-DAG operational semantics **5**
3.1 Graph order . . . 5
3.2 Configurations . . . 6
3.3 Labels, freshness, and transition rules . . . 7
3.4 Store invariant and GCA events . . . 8

4 Replay contexts and set-relative order **9**
4.1 Replay context . . . 9
4.2 Replay-order relation and set-relative linearization . . . 10
4.3 Sufficient replay laws and convergence . . . 10

5 Canonical states and canonical configurations **12**

6 Merge preservation **13**
6.1 The semantic contract . . . 13
6.2 Proof routes for merge preservation . . . 13
6.2.1 Canonical-state Join laws . . . 14
6.2.2 Universal equations as a shortcut . . . 17
6.3 Ordinary adequacy . . . 18

7 Issuance and client-facing correctness **18**
7.1 Sequential specification . . . 18
7.2 Issuance and certified execution . . . 19
7.3 Correctness obligation . . . 20

8 Canonical virtual merge bases **20**
8.1 Resolver and widened merge . . . 20
8.2 Recursive maximal-ancestor fold . . . 21
8.3 Virtual canonicity and adequacy . . . 21
8.4 Verification certificates . . . 22

9 Distributed commit-history collection **23**
9.1 Local holdings and evidence . . . 23
9.2 Compression certificate . . . 24
9.3 Asynchronous refinement . . . 25

10 Runtime and datatype-state collection **25**
10.1 Commit-store runtime refinement . . . 25
10.2 Datatype-state GC . . . 25
10.3 Composition . . . 26

11 Verification certificates and instances **26**
11.1 Formal instance profiles . . . 26
11.2 Proof routes and evidence status . . . 45

A Exact correspondence index **46**

B Historical binary boundary and negative evidence **47**

C Notation and elaboration notes **47**

D Merge-law adapters **48**

1 Purpose and evidence boundary

This document is written for readers who want to understand the formal theory before navigating its Lean implementation. It reconstructs the interface and theorem dependency graph and then instantiates them for every released RDT. It answers four questions:

1. What are the exact states and transitions of the MRDT machine?
2. What invariant makes every stored version replayable, and what obligation preserves it at merge?
3. How is internal replay connected to an independent client specification?
4. Which history- and state-collection steps refine the same uncollected semantics?

The top-level result is client-facing: every released datatype packages an issuance discipline, a merge-adequacy proof, an independent sequential specification, and a refinement proof. Consequently, every version produced by an issuance-certified ordinary or virtual-merge-base execution admits a legal sequential history with the same events and the same query results. Sections 2–8 define the layers needed by this statement. Sections 9–10 add optional history- and datatype-state collection refinements. Section 11 then presents the complete datatype packages and their collection status.

Reader's map.

Layer	Contract and consequence	Location
Operational machine	Events and the datatype signature determine fork, apply, query, and ternary merge in a version DAG.	Sections 2–3
Replay model	A finite event set admits a respecting replay under the sufficient replay laws, which also make its fold unique.	Sections 4–5
Merge preservation	An all-context or restricted merge theorem preserves a canonical state at a merge step. Execution invariants provide another proof route.	Section 6
Client correctness	Issuance, one visibility-plus-*ord* order, an independent sequential machine, and a representation relation turn internal replay into observable sequential correctness.	Section 7
Virtual bases	A recursive fold over maximal common ancestors constructs the canonical intersection state when no stored GCA exists.	Section 8
Released instances	Each package records its representation, issuance policy, merge theorem, sequential specification, relation, and proof route.	Section 11
Collection	Commit-history and datatype-state collection refine the same uncollected semantics and compose by trace erasure; the datatype profiles state what each released representation can presently reclaim.	Sections 9–10

The mathematical displays are transcriptions or notation-level expansions of the cited declarations. The trusted computing base for machine-checked statements is Lean's kernel and the axioms reported by the ledger; Mathlib supplies checked library proofs. The transcription, notation, prose, and links remain review obligations. No claim is made that a separately implemented runtime is extracted from Lean.

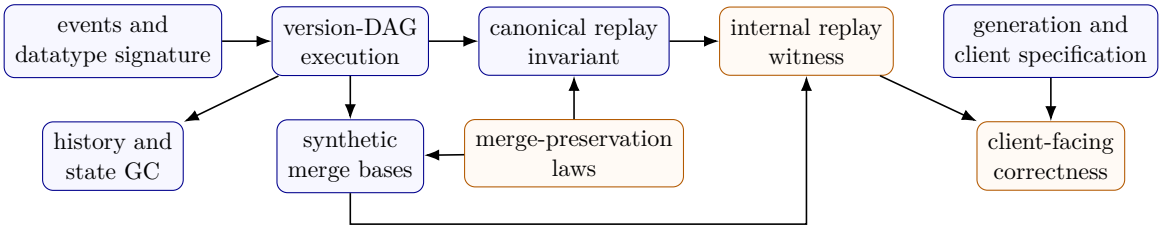


Figure 1: Semantic dependency order. Arrows point from prerequisite to consumer. The two GC layers are optional refinements, not premises of client-facing correctness.

2 Primitive types and datatype signatures

2.1 Identifiers and events

Replicas, timestamps, and versions are natural numbers:

$$\mathsf{Replica} = \mathbb{N}, \qquad \mathsf{Timestamp} = \mathbb{N}, \qquad \mathsf{Version} = \mathbb{N}.$$

For an application-operation type O, an event is

$$\mathsf{Event}(O) = \mathsf{Timestamp} \times \mathsf{Replica} \times O.$$

If $e = (t, r, o)$, write $\mathsf{time}(e) = t$ and $\mathsf{rep}(e) = r$. Timestamps are Lamport timestamps. Global uniqueness and causal monotonicity are properties of valid configurations and the apply transition, not of the bare product type.

Lean: `Foundation.Replica, Foundation.Timestamp, Foundation.Op`.

Definition 2.1 (MRDT signature)**.** An MRDT signature is

$$D = (\Sigma, \sigma_0, O, Q, R, \mathsf{do}, \mathsf{merge}, \mathsf{query})$$

with decidable equality on Σ and O, and

$$\begin{aligned} \mathsf{do} &: \Sigma \to \mathsf{Event}(O) \to \Sigma, \\ \mathsf{merge} &: \Sigma \to \Sigma \to \Sigma \to \Sigma, \\ \mathsf{query} &: \Sigma \to Q \to R. \end{aligned}$$

The intended reading of $\mathsf{merge}(l, a, b)$ is a three-way merge with ancestor state l and branch states a, b. The ancestor is semantically necessary for datatypes whose state combines by addition rather than idempotent union: the flat counter, for example, uses $\mathsf{merge}(l, a, b) = a + b - l$ so that effects already present in both branches are counted once. Ternary merge covers both these group-like states and ordinary grow-only unions with one interface. The paper's do is the Lean field `MRDTSig.update`; Lean reserves `do` for its term syntax.

Lean: `MRDTSig`.

This signature is the semantic datatype interface; some models use noncomputable queries. Verification attaches replay, merge, generation, and client certificates to this same D.

Running example: RGA. The production RGA stores grow-only insertion and deletion facts:

$$\begin{aligned} \Sigma_{\mathrm{RGA}} &= (\mathbb{N}^2 \to \mathsf{Bool}) \times (\mathbb{N} \to \mathsf{Bool}), \\ O_{\mathrm{RGA}} &= \mathsf{addAfter}(\mathit{anchor}) \mid \mathsf{remove}(\mathit{id}). \end{aligned}$$

An insertion's timestamp is its identifier. The event sets the birth bit indexed by that identifier and its anchor; a removal sets the tombstone bit for its identifier. Merge is pointwise Boolean union of the GCA and two branch states. The query sorts the finite insertion support, inserts each identifier after its anchor, and filters tombstoned identifiers. Thus the implementation state is unordered grow-only evidence, while the client observes an ordinary list.

Lean: `RGA.RGAM, RGA.rgaUpdate, RGA.read`.

2.2 Finite replay

Finite replay uses only the merge-free projection

$$D_U = (\Sigma, \sigma_0, O, \mathsf{do}).$$

Lean calls this derived structure `UpdateSig`; it retains precisely the fields used when an event history is folded.

For $\pi = [e_1, \ldots, e_n]$,

$$\mathsf{applySeq}(D, s, \pi) = \mathsf{do}(\cdots \mathsf{do}(\mathsf{do}(s, e_1), e_2) \cdots, e_n).$$

A list enumerates a set exactly when

$$\mathsf{listPermOf}(\pi, E) \iff \mathsf{Nodup}(\pi) \wedge \forall e.\ (e \in \pi \leftrightarrow e \in E).$$

It respects a relation R when no later element is required to precede an earlier one:

$$\mathsf{respects}(\pi, R) \iff \mathsf{Pairwise}_{\pi}(\lambda a\ b.\ \neg R(b, a)).$$

Lean: `Foundation.applySeq, Foundation.listPermOf, Foundation.respects`.

Concrete updates commute when applying them in either order gives the same implementation state, from every starting state:

$$e_1 \leftrightarrow e_2 \iff \forall s : \Sigma.\ \mathsf{do}(\mathsf{do}(s, e_1), e_2) = \mathsf{do}(\mathsf{do}(s, e_2), e_1).$$

The notation ord denotes an independently selected full-event replay order. The submission manuscript reserves rc for its operation-level conflict relation and event lift. An `EventOrder` may inspect event metadata and is not required to be that lift.

An event replay order P determines a directed relation on events, written $e_1 \xrightarrow{\mathsf{ord}} e_2$: replay e_1 before e_2 to resolve their conflict. Lean represents the policy by a resolver with results `Fst_then_snd`, `Snd_then_fst`, and `Either`; the directed relation is precisely `EventOrder.Before`, which tests for `Fst_then_snd`. The theory tests the reverse edge on the reversed pair. The carrier alone does not guarantee that a `Snd_then_fst` verdict agrees with that reverse test. The relation specifies the intended resolution of concurrent events; it is not defined by concrete-update non-commutation. For example, LWW resolves concurrent writes by increasing timestamp, although its concrete updates retain the maximum timestamp and commute. The default policy returns Either. Each public package selects one such policy. The sequential legality predicate may impose additional constraints on the witness list; these must also be stated in the datatype's contract. Section 4.2 derives its set-relative client relation once replay contexts have supplied visibility and the finite event set. Finite replay, commutation, and the policy are factored through the merge-free projection of D.

Lean: `Foundation.UpdateSig.commutes, Foundation.OrderResult, Foundation.EventOrder, Foundation.EventOrder.Before, Foundation.UpdateSig.replayBefore`.

Lean: `Production.registry`.

3 Version-DAG operational semantics

3.1 Graph order

For $P : \mathsf{Version} \to \mathsf{List}(\mathsf{Version})$,

$$\mathsf{Reaches}(P, a, b) \iff (\lambda x\ y.\ x \in P(y))^*(a, b).$$

The ordinary merge rule uses a greatest-common-ancestor predicate:

$$\begin{aligned}\mathsf{IsGCA}(P, v_1, v_2, v_T) \iff\ & v_T\ \mathsf{Reaches}\ v_1 \wedge v_T\ \mathsf{Reaches}\ v_2 \\ & \wedge \forall w.\ w\ \mathsf{Reaches}\ v_1 \to w\ \mathsf{Reaches}\ v_2 \to w\ \mathsf{Reaches}\ v_T.\end{aligned}$$

Every common ancestor reaches a GCA. Ranked reachability is antisymmetric, so a GCA is unique when it exists. A criss-cross DAG may instead have several incomparable maximal common ancestors and no GCA; Section 8 constructs its virtual merge base.

Lean: `Reaches, IsGCA`.

Lean: `isGCA_unique`.

For a pair of versions a, b, define

$$\begin{aligned}\mathsf{CommonAncestors}(P, a, b) &= \{x \mid x \mathsf{\ Reaches\ } a \wedge x \mathsf{\ Reaches\ } b\},\\ \mathsf{IsMaximalCommonAncestor}(P, a, b, m) &\iff m \in \mathsf{CommonAncestors}(P, a, b)\\ &\quad \wedge \forall x \in \mathsf{CommonAncestors}(P, a, b).\ m \mathsf{\ Reaches\ } x \to x = m.\end{aligned}$$

Thus m is maximal when no distinct common ancestor descends from it. The self-case is present because Reaches is reflexive. Several incomparable versions may satisfy this predicate.

Lean: `CommonAncestors, IsMaximalCommonAncestor`.

3.2 Configurations

For an MRDT D, write the version store as a partial map and let $A_C = \mathsf{dom}(S)$ be its allocated-version set. A configuration contains

$$\begin{array}{rcll} S & : & \mathsf{Version} \rightharpoonup (\Sigma \times \mathcal{P}(\mathsf{Event}(O))) & \text{allocated version contents,}\\ H & : & \mathsf{Replica} \rightharpoonup A_C & \text{active replica heads,}\\ P & : & \mathsf{Version} \to \mathsf{List}(\mathsf{Version}) & \text{parent lists,}\\ \mathsf{vis} & : & \mathsf{Event}(O) \to \mathsf{Event}(O) \to \mathsf{Prop} & \text{visibility.}\end{array}$$

Thus $C = \langle S, H, P, \mathsf{vis}\rangle$. Write

$$\begin{aligned}\mathsf{state}_C : A_C \to \Sigma, &\qquad \mathsf{state}_C(v) = \mathsf{fst}(S(v)),\\ \mathsf{events}_C : A_C \to \mathcal{P}(\mathsf{Event}(O)), &\qquad \mathsf{events}_C(v) = \mathsf{snd}(S(v)),\\ \mathsf{headState}_C = \mathsf{state}_C \circ H, &\qquad \mathsf{headEvents}_C = \mathsf{events}_C \circ H.\end{aligned}$$

The configuration's active event universe is

$$\begin{aligned}C.\mathsf{events} &\equiv \{e \mid \exists r, E.\ \mathsf{headEvents}_C(r) = E \wedge e \in E\}\\ &= \bigcup_{r \in \mathsf{dom}(H)} \mathsf{headEvents}_C(r).\end{aligned}$$

It contains the events observed at active replica heads, not the union of all historical version event sets. These functions and the active event universe are derived observations, not configuration components.

Lean: `Configuration.events`.

For any partial map f, write $f(x) = \bot \iff x \notin \mathsf{dom}(f)$.

A well-formed semantic configuration satisfies:

1. Every visibility endpoint occurs in some $\mathsf{headEvents}_C(r)$, and each head event set is closed under visibility.
2. Distinct active events have distinct timestamps; e_1 vis e_2 implies $\mathsf{time}(e_1) < \mathsf{time}(e_2)$; and distinct events from one replica are visibility-comparable.
3. Parents have smaller version numbers and $S(0) = (\sigma_0, \varnothing)$.

4. If $S(v_i) = (s_i, E_i)$ for $i \in \{1, 2, T\}$ and v_T is a GCA of v_1, v_2, then $E_T = E_1 \cap E_2$.
Thus a configuration is already an operationally well-formed state. Client safety and canonical replay are separate predicates.

Lean: `Configuration`.

Lean totalizes the two partial maps: `ver` and `head` return `Option`, while `parents` is the total function P. The `head_alloc` field implements the paper type $H : \mathsf{Replica} \rightharpoonup A_C$. The functions `Configuration.headState` and `Configuration.headEvents` implement the two partial compositions above. The store invariant in Section 3.4 connects P to A_C: every version named in a parent list must lie in A_C. Until that invariant is imposed, keeping P total avoids adding a second allocation domain.

Write $\mathsf{initConfig}(D)$ for the initial configuration. It has replica 0 at version 0, both with state σ_0 and empty event set. All other replicas and versions are absent, parent lists are empty, and visibility is empty.

Lean: `initConfig`.

3.3 Labels, freshness, and transition rules

The label constructors have explicit types

$$\begin{aligned}
\mathsf{fork} &: \mathsf{Replica} \to \mathsf{Replica} \to \mathsf{Label}(D),\\
\mathsf{apply} &: \mathsf{Timestamp} \to \mathsf{Replica} \to O \to \mathsf{Label}(D),\\
\mathsf{merge} &: \mathsf{Replica} \to \mathsf{Replica} \to \mathsf{Label}(D),\\
\mathsf{query} &: \mathsf{Replica} \to Q \to R \to \mathsf{Label}(D).
\end{aligned}$$

Fork is destination-first: $\mathsf{fork}(r_d, r_s)$.

Lean: `Label`.

Write $\mathsf{freshTime}(C, t)$ when no event at an allocated version has timestamp t, and $\mathsf{freshVer}(C, v')$ when $v' \notin A_C$. The condition $v < v'$ is a rank condition for the version DAG, not a timestamp comparison. It makes the parent relation well founded. The following rules display all changed semantic fields; other entries remain unchanged, and the resulting configuration satisfies the configuration conditions above.

Fork

$$\frac{H(r_s) = v \quad \mathsf{headState}_C(r_s) = s \quad \mathsf{headEvents}_C(r_s) = E \quad H(r_d) = \bot \quad \mathsf{freshVer}(C, v') \quad v < v'}{C \xrightarrow{\mathsf{fork}(r_d, r_s)} C\big[S(v') \mapsto (s, E), \quad H(r_d) \mapsto v', \quad P(v') \mapsto [v] \quad \big]}$$

Fork creates a fresh child of an existing source head, preserving that source as the child's parent.

Lean: `Step.fork`.

Lean: `Step.fork_copies_source, Step.fork_head_ne_root`.

Apply

$$\frac{H(r) = v \quad \mathsf{headState}_C(r) = s \quad \mathsf{headEvents}_C(r) = E \quad e = (t, r, o) \quad \mathsf{freshTime}(C, t) \quad \mathsf{freshVer}(C, v') \quad v < v'}{C \xrightarrow{\mathsf{apply}(t,r,o)} C\begin{bmatrix} S(v') \mapsto (\mathsf{do}(s, e), E \cup \{e\}), & H(r) \mapsto v', \\ P(v') \mapsto [v], & \mathsf{vis} \mapsto \mathsf{vis} \cup (E \times \{e\}) \end{bmatrix}}$$

Freshness gives globally unique timestamps. Because the resulting configuration also satisfies causal timestamp monotonicity, all events visible to e have smaller timestamps. Together these are the Lamport-clock conditions. The raw rule has no issuance premise.

Lean: `Step.apply`.

QUERY

$$\frac{\mathsf{headState}_C(r) = s \qquad z = \mathsf{query}(s, q)}{C \xrightarrow{\mathsf{query}(r,q,z)} C}$$

Query is observational.

Lean: `Step.query`.

MERGE

$$\frac{\begin{array}{c} H(r_i) = v_i \quad \mathsf{headState}_C(r_i) = s_i \quad \mathsf{headEvents}_C(r_i) = E_i \qquad (i \in \{1,2\}) \\ \mathsf{IsGCA}(P, v_1, v_2, v_T) \quad \mathsf{state}_C(v_T) = s_T \quad \mathsf{events}_C(v_T) = E_T \\ \mathsf{freshVer}(C, v_m) \quad v_1 < v_m \quad v_2 < v_m \end{array}}{C \xrightarrow{\mathsf{merge}(r_1,r_2)} C \begin{bmatrix} S(v_m) \mapsto (\mathsf{merge}(s_T, s_1, s_2), E_1 \cup E_2), & \\ H(r_1) \mapsto v_m, & P(v_m) \mapsto [v_1, v_2] \end{bmatrix}}$$

The first replica receives the merged head. Merge creates no event.

Lean: `Step.merge`.

3.4 Store invariant and GCA events

The store invariant has three components. First, define

$$\mathsf{ParentsAllocated}(S, P) \iff \forall v, p.\ p \in P(v) \Rightarrow p \in \mathsf{dom}(S).$$

Every parent named by an edge is present in the version store. Under this condition, P may be viewed as $P : \mathsf{Version} \to \mathsf{List}(A_C)$, and its restriction to allocated children has type $P|_{A_C} : A_C \to \mathsf{List}(A_C)$. Define event monotonicity along a parent edge by

$$\begin{aligned} \mathsf{EventsMonotone}(S, P) \iff \forall v, p, s_p, E_p, s_v, E_v.\ p \in P(v) \wedge S(p) = (s_p, E_p) \wedge S(v) = (s_v, E_v) \\ \Rightarrow E_p \subseteq E_v. \end{aligned}$$

Thus a child contains every event stored at its parent. Finally, define

$$\begin{aligned} \mathsf{EventOrigin}(S, P) \iff \forall v, s, E, e.\ S(v) = (s, E) \wedge e \in E \Rightarrow \\ \exists v_0, s_0, E_0.\ S(v_0) = (s_0, E_0) \wedge e \in E_0 \\ \wedge \forall w, s_w, E_w.\ S(w) = (s_w, E_w) \wedge e \in E_w \\ \Rightarrow \mathsf{Reaches}(P, v_0, w). \end{aligned}$$

Every stored event therefore has an allocated origin version containing it; that version is an ancestor of every allocated version containing the event. No uniqueness of the origin is asserted. The store invariant is the conjunction

$$\begin{aligned} \mathsf{StoreInv}(S, P) \iff \mathsf{ParentsAllocated}(S, P) \\ \wedge\ \mathsf{EventsMonotone}(S, P) \\ \wedge\ \mathsf{EventOrigin}(S, P). \end{aligned}$$

These three predicates correspond respectively to Lean's `parents_alloc`, `events_mono`, and `origin` fields. The proof of the GCA event equation factors through their conjunction. Its principal consequence is

$$\begin{aligned}&\mathsf{StoreInv}(S,P) \wedge \mathsf{IsGCA}(P, v_1, v_2, v_T)\\ \wedge S(v_i) = (s_i, E_i)\ (i \in \{1,2,T\}) &\Longrightarrow E_T = E_1 \cap E_2.\end{aligned}$$

Lean: `StoreInv, gca_events_of_storeInv`.

4 Replay contexts and set-relative order

4.1 Replay context

For a stored event set E, the replay problem is to enumerate exactly the events in E in an admissible order and fold their updates from σ_0. Admissibility depends on which events have been observed and on their visibility relation. It does not depend on version identifiers, parent edges, materialized states, or merge bases. The replay context isolates this smaller input to the ordering proofs.

Let $L : \mathsf{Replica} \rightharpoonup \mathcal{P}(\mathsf{Event}(O))$ map each active replica to the event set at its current head. Here a vis b means that a is visible to b, so a causally precedes b. Define the observed event universe by

$$\mathsf{Observed}_L(e) \Longleftrightarrow \exists r, E.\ L(r) = E \wedge e \in E.$$

The two required coherence conditions are

$$\begin{aligned}\mathsf{TimestampUnique}(L) &\Longleftrightarrow \forall a, b.\ \mathsf{Observed}_L(a) \wedge \mathsf{Observed}_L(b) \wedge a \neq b \Rightarrow \mathsf{time}(a) \neq \mathsf{time}(b),\\ \mathsf{SameReplicaTotal}(L, \mathsf{vis}) &\Longleftrightarrow \forall a, b.\ \mathsf{Observed}_L(a) \wedge \mathsf{Observed}_L(b) \wedge a \neq b\\ &\qquad \wedge \mathsf{rep}(a) = \mathsf{rep}(b) \Rightarrow a\ \mathsf{vis}\ b \vee b\ \mathsf{vis}\ a.\end{aligned}$$

Thus timestamps identify observed events globally, while visibility compares every pair of distinct events issued by one replica. A replay context is formally

$$\mathsf{ReplayContext}(D) = \left\langle \begin{array}{l} L : \mathsf{Replica} \rightharpoonup \mathcal{P}(\mathsf{Event}(O)),\\ \mathsf{vis}\colon \mathsf{Event}(O) \to \mathsf{Event}(O) \to \mathsf{Prop},\\ u_{\mathrm{ts}} : \mathsf{TimestampUnique}(L),\\ u_{\mathrm{rep}} : \mathsf{SameReplicaTotal}(L, \mathsf{vis}) \end{array} \right\rangle.$$

For a replay context $R = \langle L, \mathsf{vis}, u_{\mathrm{ts}}, u_{\mathrm{rep}} \rangle$, the event universe is the union of the active replicas' head-event sets:

$$R.\mathsf{events} = \{e \mid \mathsf{Observed}_L(e)\} = \bigcup_{r \in \mathsf{dom}(L)} L(r).$$

Every MRDT configuration C projects to a replay context by taking $L = \mathsf{headEvents}_C$ and retaining $C.\,\mathsf{vis}$. The configuration fields `timestamps_distinct` and `vis_total_same_replica` supply the two proofs. Lean totalizes L as an Option-valued function; `ReplayContext.L` receives the derived head-event map, and `Configuration.replayContext` constructs the projection. Write $R_C = \mathsf{replayContext}(C)$ for this projection whenever C is an MRDT configuration. The projection preserves the active event universe:

$$R_C.\mathsf{events} = C.\mathsf{events}.$$

Lean: `Foundation.ReplayContext`.

Lean: `Configuration.replayContext`.

4.2 Replay-order relation and set-relative linearization

Fix an event replay order P and its directed relation *ord* as defined in Section 2. The relation expresses the datatype's resolution policy independently of its implementation. For concurrent LWW writes with distinct timestamps,

$$a \xrightarrow{\mathsf{ord}} b \iff \mathsf{time}(a) < \mathsf{time}(b).$$

The implementation retains the maximum timestamp regardless of delivery order; an ordinary overwrite replay places the smaller timestamp first.

The replay-order relation now supplies the directed policy edge used by linearization. For replay context R, version event set E, and events e_1, e_2, define the set-relative linearization relation by

$$\begin{aligned} e_1 \ \mathsf{loOn}_{R,E}\ e_2 \iff & (e_1 \ \mathsf{vis}\ e_2 \wedge (e_1 \xrightarrow{\mathsf{ord}} e_2 \vee e_2 \xrightarrow{\mathsf{ord}} e_1)) \\ & \vee (\neg(e_1 \ \mathsf{vis}\ e_2) \wedge \neg(e_2 \ \mathsf{vis}\ e_1) \wedge e_1 \xrightarrow{\mathsf{ord}} e_2 \\ & \wedge \neg\exists e_3 \in E.\ e_2 \ \mathsf{vis}\ e_3 \wedge (e_2 \xrightarrow{\mathsf{ord}} e_3 \vee e_3 \xrightarrow{\mathsf{ord}} e_2)). \end{aligned}$$

An event $e_3 \in E$ satisfying the final existential condition is an *absorber* for e_2. Only an absorber inside E can remove an arbitration edge needed to replay E. Growing the set may add absorbers and therefore remove such edges:

$$E \subseteq E' \wedge a \ \mathsf{loOn}_{R,E'}\ b \Longrightarrow a \ \mathsf{loOn}_{R,E}\ b.$$

The historical global order searches for absorbers without the membership restriction. Every historical edge is an edge of $\mathsf{loOn}_{R,E}$. It agrees with $\mathsf{loOn}_{R,R.\mathsf{events}}$ when visibility targets belong to $R.\mathsf{events}$, as they do for a configuration's replay context. The set parameter is therefore essential in the general theory: whether an arbitration edge is needed depends on whether its absorber belongs to the particular version being replayed, not merely on the global event universe.

Lean: `Foundation.loOn, Foundation.loOn_mono, Foundation.loOn_of_lo`.

For an all-Either policy, *ord* has no edges and therefore

$$\neg(a \ \mathsf{loOn}_{R,E}\ b).$$

In particular, Either does not mean "retain all visibility." Visibility contributes to loOn only for a pair that *ord* classifies as a conflict.

RGA instance. RGA insertion and tombstone updates commute at the raw storage layer, but its ordinary-list specification still has conflicts. Its sole *ord* relation orders the RGA conflicts described in Section 11; loOn uses that relation directly.

Lean: `RGA.replayOrder`.

4.3 Sufficient replay laws and convergence

The following optional algebraic laws are one sufficient route to replay convergence under P. They are not the definition of *ord* and are not required fields of the public datatype certificate. In this subsection and the replay theorems below, D and P are fixed together.

Write

$$\mathsf{distinct}(a, b) \iff \mathsf{time}(a) \neq \mathsf{time}(b).$$

Replay contexts make this equivalent to event inequality on observed events. The update-layer bundle has three conjuncts. Coverage of concrete noncommuting pairs is

$$\mathsf{NoncommCovered}(D, P) \iff \forall a, b.\ \neg(a \leftrightarrow b) \implies (a \xrightarrow{\mathsf{ord}} b \lor b \xrightarrow{\mathsf{ord}} a).$$

This is a one-way proof obligation: a pair not ordered in either direction must commute concretely. An ordered pair may also commute. In particular, all-commuting updates satisfy coverage for every choice of *ord*. Write $Q^+(a, b)$ when there is a nonempty finite Q-path from a to b, and define

$$\mathsf{Acyclic}(Q) \iff \forall a.\ \neg Q^+(a, a).$$

Replay-order acyclicity is

$$\mathsf{ReplayAcyclic}(D, P) \iff \mathsf{Acyclic}(\lambda a\, b.\ a \xrightarrow{\mathsf{ord}} b).$$

It permits policy chains of arbitrary finite length and rules out only directed cycles. Finally, conditional commutation lift is

$$\begin{aligned}\mathsf{CondCommLift}(D, P) \iff \forall s, e, e', e'', \pi.\ & \mathsf{distinct}(e, e') \land \mathsf{distinct}(e, e'') \land \mathsf{distinct}(e', e'') \\ & \land e \xrightarrow{\mathsf{ord}} e' \land (e' \xrightarrow{\mathsf{ord}} e'' \lor e'' \xrightarrow{\mathsf{ord}} e') \\ & \implies \mathsf{do}(\mathsf{applySeq}(D, \mathsf{do}(\mathsf{do}(s, e'), e), \pi), e'') \\ & \quad = \mathsf{do}(\mathsf{applySeq}(D, \mathsf{do}(\mathsf{do}(s, e), e'), \pi), e'').\end{aligned}$$

Therefore

$$\mathsf{ReplayLaws}(D, P) \iff \mathsf{NoncommCovered}(D, P) \land \mathsf{ReplayAcyclic}(D, P) \land \mathsf{CondCommLift}(D, P).$$

These conjuncts correspond respectively to Lean's `noncomm_covered`, `replayOrder_acyclic`, and `cond_comm_lift` fields. The conditional swap uses the *ord*-related successor tested by the linearization relation above; that successor need not be concretely noncommuting. When all concrete updates commute, both swap orders already agree before the successor is applied. Thus every acyclic *ord* satisfies these sufficient laws for an all-commuting implementation.

Lean: `ReplayAcyclic, ReplayLaws, ReplayLaws.of_all_comm`.

With these laws, transitive and irreflexive visibility, and finite support, $\mathsf{loOn}_{R,E}$ is acyclic on E, admits a respecting enumeration, and all respecting enumerations fold to the same state. Intuitively, acyclicity says that the replay constraints are jointly schedulable; existence produces at least one such schedule; and convergence says that choosing a different valid schedule cannot change the result. Thus a finite event set E determines one replayed state even when it admits several valid orders.

For events in E, write

$$a\ \mathsf{loOn}^{\neq}_{R,E}\ b \iff a \neq b \land a \in E \land b \in E \land a\ \mathsf{loOn}_{R,E}\ b.$$

Lemma 4.1 (Existence and uniqueness of set-relative replay)**.** *Suppose*

$$\mathsf{ReplayAcyclic}(D, P), \qquad \mathsf{Transitive}(R.\mathsf{vis}), \qquad \mathsf{Irreflexive}(R.\mathsf{vis}),$$
$$E \subseteq R.\mathsf{events}, \qquad \mathsf{listPermOf}(\ell, E).$$

Then:

1. $\mathsf{loOn}^{\neq}_{R,E}$ *is acyclic:*

$$\mathsf{Acyclic}(\lambda a\, b.\ a\ \mathsf{loOn}^{\neq}_{R,E}\ b).$$

2. *E has a respecting enumeration:*

$$\exists \rho.\ \mathsf{listPermOf}(\rho, E) \wedge \mathsf{respects}(\rho, \mathsf{loOn}_{R,E}).$$

3. *if* $\mathsf{ReplayLaws}(D, P)$ *also holds, all respecting enumerations of E have the same fold, from every starting state:*

$$\begin{aligned}\forall s, \rho_1, \rho_2.\quad & \mathsf{listPermOf}(\rho_1, E) \wedge \mathsf{listPermOf}(\rho_2, E) \\ & \wedge \mathsf{respects}(\rho_1, \mathsf{loOn}_{R,E}) \wedge \mathsf{respects}(\rho_2, \mathsf{loOn}_{R,E}) \\ & \Longrightarrow \mathsf{applySeq}(D, s, \rho_1) = \mathsf{applySeq}(D, s, \rho_2).\end{aligned}$$

Lean: `loOnNe_acyclic_of_replayAcyclic, convergence_on_of_replayLaws`.

Lean: `exists_loOn_respecting_perm_of_replayAcyclic`.

5 Canonical states and canonical configurations

Definition 5.1 (Canonical state)**.**

$$\begin{aligned}\mathsf{Canonical}(R, E, s) \Longleftrightarrow \exists \rho.\ & \mathsf{listPermOf}(\rho, E) \wedge \mathsf{respects}(\rho, \mathsf{loOn}_{R,E}) \\ & \wedge \mathsf{applySeq}(D, \sigma_0, \rho) = s.\end{aligned}$$

This is an existence predicate, not a uniqueness assertion. Under the replay laws, transitive and irreflexive visibility, and $E \subseteq R.\mathsf{events}$, set-relative convergence makes every respecting enumeration have the same fold.

Lean: `Foundation.IsCanonicalState`.

Definition 5.2 (Canonical configuration)**.** Let $R_C = \mathsf{replayContext}(C)$. Then $\mathsf{CanonicalConfig}(C)$ consists of:

canonical :	$S(v) = (s, E) \Rightarrow \mathsf{Canonical}(R_C, E, s)$,
visTrans :	$a\ \mathsf{vis}\ b \wedge b\ \mathsf{vis}\ c \Rightarrow a\ \mathsf{vis}\ c$,
visIrrefl :	$\neg(a\ \mathsf{vis}\ a)$,
supported :	$S(v) = (s, E) \wedge a \in E \Rightarrow a \in C.\mathsf{events}$,
causal :	$S(v) = (s, E) \wedge a\ \mathsf{vis}\ b \wedge b \in E \Rightarrow a \in E$,

Lean: `CanonicalConfig`.

The invariant ranges over every allocated version, including historical merge bases. Its internal replay projection is

$$\begin{aligned}\mathsf{HasReplayWitness}(C) \Longleftrightarrow \forall v, s, E.\ S(v) = (s, E) \rightarrow \exists \pi.\ & \mathsf{listPermOf}(\pi, E) \\ & \wedge \mathsf{respects}(\pi, \mathsf{loOn}_{R_C,E}) \\ & \wedge \mathsf{applySeq}(D, \sigma_0, \pi) = s.\end{aligned}$$

Canonicality supplies this witness directly, with the same selected *ord* and the same set-relative order:

$$\mathsf{CanonicalConfig}(C) \Longrightarrow \mathsf{HasReplayWitness}(C).$$

This establishes internal replay adequacy. Section 7 relates the replay witness to an independent client specification.

Lean: `HasReplayWitness, hasReplayWitness_of_canonical`.

6 Merge preservation

Fix D and its event replay order P throughout this section. The notation $\mathsf{Canonical}(R, E, s)$ uses that same policy. We write P explicitly in the Join contracts; Lean supplies it as an implicit `EventOrder` argument.

6.1 The semantic contract

For replay context R, abbreviate

$$\begin{aligned}\mathsf{Context}(R) &\equiv \mathsf{Transitive}(\mathsf{vis}) \wedge \mathsf{Irreflexive}(\mathsf{vis}),\\ \mathsf{Supported}_R(E) &\equiv E \subseteq R.\mathsf{events},\\ \mathsf{WeakClosed}_R(E) &\equiv \forall a, b.\ a\ \mathsf{vis}\ b \wedge \neg(a \leftrightarrow b) \wedge b \in E \to a \in E.\end{aligned}$$

Weak closure requires retention of noncommuting visible predecessors, while allowing subsets that omit commuting predecessors. This enlarges the scope of the merge theorem beyond causally closed histories. Commutation permits swapping events, not erasing them: two increments commute, but both still contribute to the fold.

Definition 6.1 (Join at a context)**.** `JoinAt D R` is

$$\begin{aligned}\forall E_1, E_2, s_0, s_1, s_2.\ \mathsf{Context}(R) \wedge \bigwedge_{i=1,2} (\mathsf{Supported}_R(E_i) \wedge \mathsf{WeakClosed}_R(E_i))\\ \wedge\ \mathsf{Canonical}(R, E_1 \cap E_2, s_0) \wedge \mathsf{Canonical}(R, E_1, s_1) \wedge \mathsf{Canonical}(R, E_2, s_2)\\ \Longrightarrow \mathsf{Canonical}(R, E_1 \cup E_2, \mathsf{merge}(s_0, s_1, s_2)).\end{aligned}$$

`Join D` is $\forall R$. `JoinAt D R`.

Lean: `JoinAt, Join`.

Join is a sufficient merge theorem: canonical intersection and branch states map to a canonical union state. Its context-local statement is reused by both ordinary and virtual-base executions.

Definition 6.2 (Restricted merge scope)**.** For a predicate $G : \mathsf{ReplayContext}(D) \to \mathsf{Prop}$,

$$\mathsf{JoinOn}(D, P, G) \Longleftrightarrow \forall R.\ G(R) \to \mathsf{JoinAt}(D, P, R).$$

Thus all-context `Join` has no configuration precondition, whereas `JoinOn` records one explicitly without saying how executions establish it. All-context Join supplies Join under every predicate, and $\mathsf{JoinOn}(D, P, \lambda_.\mathsf{True})$ is equivalent to $\mathsf{Join}(D, P)$.

Lean: `JoinOn, Join.toJoinOn, joinOn_true_iff`.

These contracts support the reusable merge-preservation proof. A datatype may instead prove replay adequacy by an execution invariant, relating branch states more tightly than arbitrary respecting replays do. The efficient OR-set uses this route. The remaining definitions in this section are sufficient ways to prove all-context `Join`, not obligations imposed on every public package.

6.2 Proof routes for merge preservation

The reusable proof has one primary canonical-state contract. A datatype may prove that contract directly or obtain it from stronger equations over arbitrary states. It may also bypass the reusable induction and prove `Join` or `JoinOn` directly.

6.2.1 Canonical-state Join laws

For any nonempty finite family of event sets, write

$$\mathsf{Admissible}_R(E_1,\dots,E_n) \equiv \mathsf{Context}(R) \wedge \bigwedge_{i=1}^{n} (\mathsf{Supported}_R(E_i) \wedge \mathsf{WeakClosed}_R(E_i)),$$
$$\mathsf{Maximal}_{R,U}(e) \equiv e \in U \wedge \forall x \in U.\ x \neq e \Rightarrow \neg(e\ \mathsf{loOn}_{R,U}\ x).$$

Define the one-step noncommuting-visibility relation by $\mathsf{visNC}_R(a,b) \equiv a\ \mathsf{vis}\ b \wedge \neg(a \leftrightarrow b)$, and write visNC_R^+ for its positive transitive closure. The principal downset of e is

$$\downarrow_R e \equiv \{x \mid x = e \vee \mathsf{visNC}_R^+(x,e)\}.$$

We omit the subscript R below when the replay context is fixed. In particular, if $e \in E$ and $\mathsf{WeakClosed}_R(E)$, then $\downarrow_R e \subseteq E$.

The common core is

$$\mathsf{JoinCoreLaws}(D) = \langle \mathsf{replay} : \mathsf{ReplayLaws}(D,P),\ \mathsf{mergeComm} : \forall l,a,b.\ \mathsf{merge}(l,a,b) = \mathsf{merge}(l,b,a)\rangle.$$

Thus `JoinCoreLaws` contains replay and branch commutativity.

Lean: `JoinCoreLaws`.

The canonical delta component, `FeasibleDeltaLaws`, contains a unit law and the two delta equations restricted to the exact canonical tuples consumed by the Join induction. Below, R ranges over replay contexts, E, E_1, E_2, U over event sets, e over events, and $s, l, m, x, y, x_0, x_1, x_2$ over states.

$$\mathsf{init}: \quad \forall R,E,s.\ \ \mathsf{Supported}_R(E) \wedge \mathsf{Canonical}(R,E,s) \Longrightarrow \mathsf{merge}(\sigma_0,\sigma_0,s) = s.$$

Thus the unit equation is required only for a state produced by a canonical replay of a supported event set.

For an event present only on the first branch,

$$\begin{aligned}
&\mathsf{localRedistribute}: \\
&\forall R,E_1,E_2,l,m,x,y,e. \\
&\mathsf{Admissible}_R(E_1,E_2) \wedge e \in E_1 \setminus E_2 \wedge \mathsf{Maximal}_{R,E_1\cup E_2}(e) \\
&\wedge \mathsf{Canonical}(R,E_1\cap E_2,l) \wedge \mathsf{Canonical}(R,{\downarrow}\, e \setminus \{e\},m) \\
&\wedge \mathsf{Canonical}(R,E_1\setminus\{e\},x) \wedge \mathsf{Canonical}(R,E_2,y) \\
&\Longrightarrow \mathsf{merge}(l,\mathsf{merge}(m,x,\mathsf{do}(m,e)),y) \\
&\qquad = \mathsf{merge}(m,\mathsf{merge}(l,x,y),\mathsf{do}(m,e)).
\end{aligned}$$

Intuitively, the equation extracts the maximal local event through the outer merge. The states l, m, x, and y represent respectively the branch intersection, the noncommuting dependency past of e, the first branch without e, and the second branch.

Figure 2 presents the two sides of `localRedistribute` as alternative factorizations of the same union history.

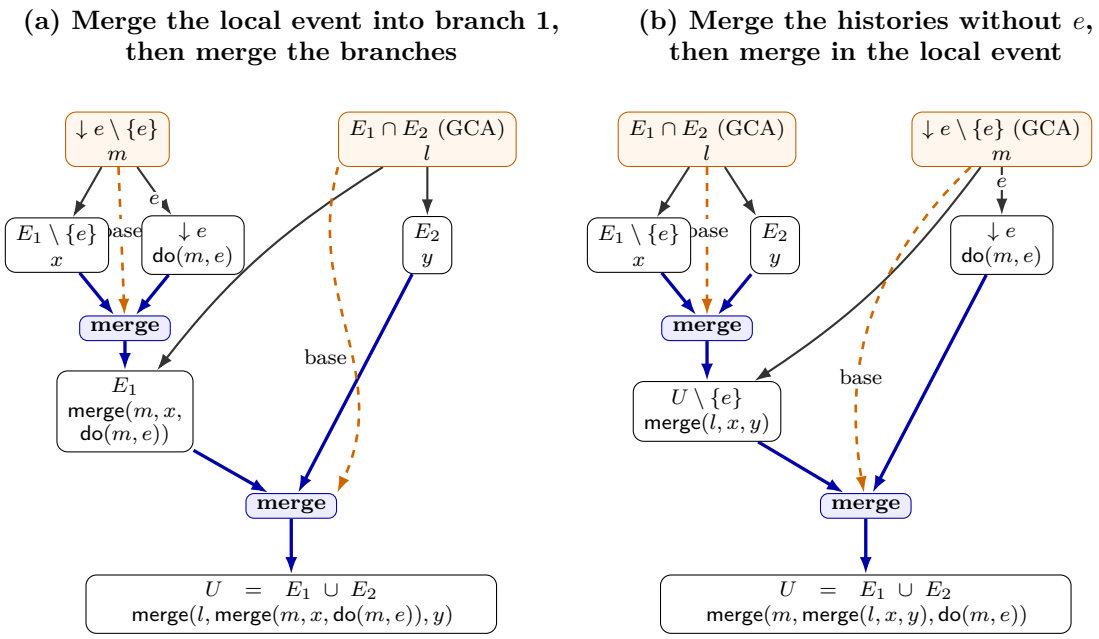


Figure 2: The local redistribution law. Each box pairs an abstract event set (top) with its canonical state (bottom). Black arrows show event-history extension. Thick blue arrows enter and leave explicit merge gates. Orange dashed arrows supply the GCA state to a gate, and orange boxes identify those GCA states. All boxes are proof-level canonical states; the law does not require all of them to be allocated versions in one execution. Panel (a) constructs the E_1 state before the outer merge. Panel (b) first constructs the $U \setminus \{e\}$ state and then merges in e. Both panels flow from top to bottom. `localRedistribute` identifies the two terminal states.

For an event shared by both branches,

$$
\begin{aligned}
&\textsf{redistribute}: \\
&\forall R, E_1, E_2, m, x_0, x_1, x_2, e. \\
&\textsf{Admissible}_R(E_1, E_2) \wedge e \in E_1 \cap E_2 \wedge \textsf{Maximal}_{R, E_1 \cup E_2}(e) \\
&\wedge \textsf{Canonical}(R, (E_1 \cap E_2) \setminus \{e\}, x_0) \wedge \textsf{Canonical}(R, {\downarrow} e \setminus \{e\}, m) \\
&\wedge \textsf{Canonical}(R, E_1 \setminus \{e\}, x_1) \wedge \textsf{Canonical}(R, E_2 \setminus \{e\}, x_2) \\
&\implies \textsf{merge}(\textsf{merge}(m, x_0, \textsf{do}(m, e)), \textsf{merge}(m, x_1, \textsf{do}(m, e)), \textsf{merge}(m, x_2, \textsf{do}(m, e))) \\
&\qquad = \textsf{merge}(m, \textsf{merge}(x_0, x_1, x_2), \textsf{do}(m, e)).
\end{aligned}
$$

This equation extracts one shared maximal event from all three merge components. The feasibility restrictions exclude representation states that no event history constructs.

Lean: `FeasibleDeltaLaws`.

Figure 3 presents the two sides of `redistribute` as alternative factorizations of the same union history.

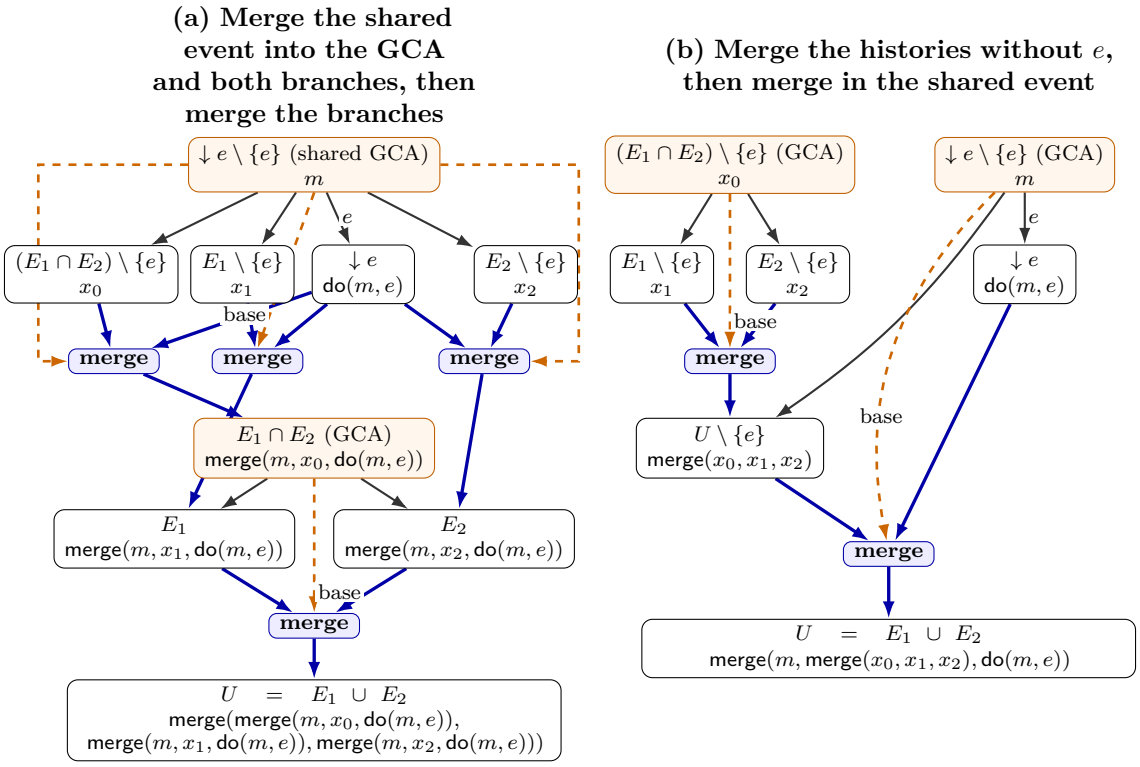


Figure 3: The shared redistribution law. Each box pairs an abstract event set (top) with its canonical state (bottom). Black arrows show event-history extension. Thick blue arrows enter and leave explicit merge gates. Orange dashed arrows supply the GCA state, and orange boxes identify GCA states. Panel (a) uses the same state m as the base of three inner merges, producing the branch intersection and the two branch states before the outer merge. Panel (b) first constructs $U \setminus \{e\}$ at base x_0, then merges it with $\downarrow e$ at base m. The fan-out edges in panel (a) reuse the same m and $\textsf{do}(m, e)$ states. Both panels flow from top to bottom, and `redistribute` identifies their terminal states.

The last component peels the selected maximal event from the target union.

$$\begin{aligned}\mathsf{CausalDeltaLaw}(D) \iff \forall R, U, A, B, e.\quad & \mathsf{Admissible}_R(U) \wedge \mathsf{Maximal}_{R,U}(e) \\ & \wedge \mathsf{Canonical}(R, U \setminus \{e\}, A) \wedge \mathsf{Canonical}(R, {\downarrow} e \setminus \{e\}, B) \\ & \implies \mathsf{merge}(B, A, \mathsf{do}(B, e)) = \mathsf{do}(A, e).\end{aligned}$$

Here A represents the remaining history and B represents the event's noncommuting dependency past.

Lean: `CausalDeltaLaw`.

The single instance-facing bundle is

$$\begin{aligned}\mathsf{CanonicalJoinLaws}(D) = \langle & \mathsf{core} : \mathsf{JoinCoreLaws}(D), \\ & \mathsf{delta} : \mathsf{FeasibleDeltaLaws}(D), \\ & \mathsf{causalDelta} : \mathsf{CausalDeltaLaw}(D)\rangle.\end{aligned}$$

Its primary theorem is

$$\mathsf{CanonicalJoinLaws}(D) \implies \mathsf{Join}(D, P).$$

The proof inducts on the union history. The empty case uses the canonical unit law. A maximal event local to one branch uses `localRedistribute`; a maximal event shared by both branches uses `redistribute`. Both cases use `CausalDeltaLaw` to peel the selected event.

Lean: `CanonicalJoinLaws`.

Lean: `CanonicalJoinLaws.join, JoinProof.ofFeasibleStateLaws`.

6.2.2 Universal equations as a shortcut

Many datatypes satisfy stronger equations for arbitrary representation states. The universal merge bundle `MergeLaws` contains only the fields used by the universal-to-canonical bridge:

$$\begin{aligned}\mathsf{MergeLaws}(D) = \langle & \mathsf{replay} : \mathsf{ReplayLaws}(D, P), \\ & \mathsf{mergeComm} : \forall l, a, b.\ \mathsf{merge}(l, a, b) = \mathsf{merge}(l, b, a), \\ & \mathsf{mergeInit} : \forall s.\ \mathsf{merge}(\sigma_0, \sigma_0, s) = s\rangle.\end{aligned}$$

Lean: `MergeLaws`.

The bundle `DeltaLaws` contains the two universal delta equations. Here c is an arbitrary state; the canonical adapter substitutes $\mathsf{do}(m, e)$ for it.

$$\begin{aligned}\mathsf{merge}(\mathsf{merge}(m, x_0, c), \mathsf{merge}(m, x_1, c), \mathsf{merge}(m, x_2, c)) &= \mathsf{merge}(m, \mathsf{merge}(x_0, x_1, x_2), c), \\ \mathsf{merge}(l, \mathsf{merge}(m, x, c), y) &= \mathsf{merge}(m, \mathsf{merge}(l, x, y), c).\end{aligned}$$

Lean: `DeltaLaws`.

The auxiliary `CommutingPeelLaw` states that an update can be pulled through a merge when it commutes with every event replayed in the other two components:

$$\begin{aligned}&\mathsf{CommutingPeelLaw}(D) \iff \\ &\forall a, e, \pi_0, \pi_2.\quad (\forall x \in \pi_0.\ e \leftrightarrow x) \wedge (\forall x \in \pi_2.\ e \leftrightarrow x) \\ &\implies \mathsf{merge}\big(\mathsf{applySeq}(D, \sigma_0, \pi_0), \mathsf{do}(a, e), \mathsf{applySeq}(D, \sigma_0, \pi_2)\big) \\ &\quad = \mathsf{do}\Big(\mathsf{merge}\big(\mathsf{applySeq}(D, \sigma_0, \pi_0), a, \mathsf{applySeq}(D, \sigma_0, \pi_2)\big), e\Big).\end{aligned}$$

This auxiliary adapter sits outside the canonical Join contract. Together with `MergeLaws` and universal update commutativity, it supplies `CausalDeltaLaw` for the all-commuting instance class.

$$\mathsf{MergeLaws}(D) + \mathsf{CommutingPeelLaw}(D) + (\forall e, e'.\ e \leftrightarrow e') \Longrightarrow \mathsf{CausalDeltaLaw}(D).$$

Lean: `CommutingPeelLaw`.

Lean: `causalDeltaLaw_of_all_comm`.

RGA satisfies these universal equations by pointwise Boolean identities. The adapter details are in Appendix D; the resulting merge theorem holds for every replay context. Issuance is used separately to construct the legal ordinary-list witness.

Lean: `RGA.join`.

6.3 Ordinary adequacy

The initial configuration is canonical. Fork copies a canonical version; apply extends one with a fresh maximal event; query stutters; and merge is the Join implication after the GCA theorem identifies the base set with $E_1 \cap E_2$. Write $\mathsf{Reachable}(\mathsf{Step}, C_0, C)$ for the reflexive-transitive closure of the raw transition relation. Hence

$$\mathsf{Join}(D, P) \to \mathsf{Reachable}(\mathsf{Step}, \mathsf{initConfig}(D), C) \to \mathsf{HasReplayWitness}(C).$$

The proof establishes `CanonicalConfig C` first and projects replay only at the end.

Lean: `replayWitness_of_join`.

7 Issuance and client-facing correctness

Fix a datatype D with operations O, concrete state Σ, query type Q, and result type R, and an event replay order P supplying its replay-order relation *ord*. We first define the sequential behavior against which D is checked, then the executions to which the correctness claim applies.

7.1 Sequential specification

A sequential specification *Spec* contains

$$\begin{aligned} \Sigma_S &: \mathsf{Type}, \\ \sigma_0^S &: \Sigma_S, \\ \mathsf{step}_S &: \Sigma_S \to \mathsf{Event}(O) \to \Sigma_S, \\ \mathsf{Legal}_S &: \mathsf{List}(\mathsf{Event}(O)) \to \mathsf{Prop}, \\ \mathsf{query}_S &: \Sigma_S \to Q \to R. \end{aligned}$$

Its run is the left fold of step_S from σ_0^S. The predicate Legal_S constrains the event list; it receives neither the implementation state nor visibility. The witness must separately respect loOn, defined in Section 4.2. Legality may further constrain its order: TreeMove, for example, requires a key-sorted list.

Lean: `SequentialMachine, SequentialSpec, SequentialSpec.run`.

Choose a representation relation $\mathsf{Rel} : \Sigma \to \Sigma_S \to \mathsf{Prop}$. For a configuration C, write S for its version store and R_C for its replay context, as in the preceding sections.

Definition 7.1 (Client-facing linearizability)**.** $\mathsf{IsSpecLinearizable}(D, P, \mathit{Spec}, \mathsf{Rel}, C)$ means

$$\begin{aligned}\forall v, s, E.\ S(v) = (s, E) \to \exists \pi.\ & \mathsf{listPermOf}(\pi, E)\\ & \wedge \mathsf{respects}(\pi, \lambda e_1\, e_2.\ e_1\ \mathsf{loOn}_{R_C, E}\ e_2)\\ & \wedge \mathit{Spec}.\mathsf{Legal}(\pi)\\ & \wedge \mathsf{Rel}(s, \mathit{Spec}.\mathsf{run}(\pi))\\ & \wedge \forall q.\ D.\mathsf{query}(s, q) = \mathit{Spec}.\mathsf{query}(\mathit{Spec}.\mathsf{run}(\pi), q).\end{aligned}$$

Here loOn uses the fixed policy P.

Lean: `IsSpecLinearizable`.

For each stored version, one and the same list must enumerate exactly its events, respect loOn, satisfy the sequential legality condition, relate the two states, and give the same answer to every query. The specification and relation are part of the datatype's contract, not consequences of a replay proof.

7.2 Issuance and certified execution

An issuance policy I contains

$$I.\mathsf{CanIssue} : \mathsf{Event}(O) \to \Sigma \to \mathsf{Prop}.$$

It states when an event may be created at an issuing replica's concrete state. For configurations C, C' and a transition label ℓ, $\mathsf{IssuedStep}(D, I, C, \ell, C')$ is defined by the following rules, where H and S are the head map and version store of C:

$$\frac{\mathsf{Step}(D, C, \ell, C') \qquad \forall t, r, o.\ \ell \neq \mathsf{apply}(t, r, o)}{\mathsf{IssuedStep}(D, I, C, \ell, C')}\ \textsc{Non-apply}$$

$$\frac{H(r) = v \qquad S(v) = (s, E) \qquad I.\mathsf{CanIssue}((t, r, o), s) \qquad \mathsf{Step}(D, C, \mathsf{apply}(t, r, o), C')}{\mathsf{IssuedStep}(D, I, C, \mathsf{apply}(t, r, o), C')}\ \textsc{Apply}$$

Fork, merge, and query add no issuance check. Apply checks the actual issuing head state, in addition to the raw transition's premises.

Lean: `Issuance, IssuedStep`.

A separate condition, MintHonest, requires each event to be issuable after some visibility-respecting enumeration of its causal past:

$$\begin{aligned}&\mathsf{MintHonest}(D, I.\mathsf{CanIssue}, C) \iff\\ &\quad \forall e \in C.\mathsf{events}.\ \exists \pi.\ \mathsf{listPermOf}(\pi, \{e' \in C.\mathsf{events} \mid e'\ \mathsf{vis}\ e\})\\ &\qquad \wedge \mathsf{respects}(\pi, \mathsf{vis}) \wedge I.\mathsf{CanIssue}(e, \mathsf{applySeq}(D, \sigma_0, \pi)).\end{aligned}$$

This is an existential condition on the causal past, not a check at the stored issuing head. Its enumeration respects full visibility, whereas the client witness above respects loOn.

Lean: `MintHonest`.

Certified reachability $\mathsf{MintCertifiedReach}(D, I, C)$ has an initial case $\mathsf{MintCertifiedReach}(D, I, \mathsf{initConfig}(D))$ and the step rule

$$\frac{\mathsf{MintCertifiedReach}(D, I, C) \qquad \mathsf{IssuedStep}(D, I, C, \ell, C') \qquad \mathsf{MintHonest}(D, I.\mathsf{CanIssue}, C) \qquad \mathsf{MintHonest}(D, I.\mathsf{CanIssue}, C')}{\mathsf{MintCertifiedReach}(D, I, C')}.$$

Thus the definition requires mint honesty before and after each step. It does not derive this condition merely from the local issuance checks. A claim covering all executions that obey those checks would additionally need to establish this provenance condition.

Lean: `MintCertifiedReach`.

Erasing these premises recovers raw reachability:

$$\mathsf{MintCertifiedReach}(D, I, C) \Longrightarrow \mathsf{Reachable}(\mathsf{Step}, \mathsf{initConfig}(D), C).$$

Lean: `MintCertifiedReach.toReachable`.

7.3 Correctness obligation

For the chosen D, P, I, *Spec*, and Rel, the ordinary-execution correctness obligation is

$$\forall C.\ \mathsf{MintCertifiedReach}(D, I, C) \Longrightarrow \mathsf{IsSpecLinearizable}(D, P, \mathit{Spec}, \mathsf{Rel}, C).$$

This implication must be proved for the chosen contract; it does not follow from the interface definitions alone. In particular, $\mathsf{HasReplayWitness}(C)$ only provides an ordering whose concrete fold is the stored state. It does not by itself establish sequential legality, the representation relation, or agreement with the abstract queries.

The next section extends execution to synthetic merge bases. Section 8.4 packages the proofs for both execution semantics and describes sufficient proof routes.

8 Canonical virtual merge bases

An execution DAG need not have a GCA for every pair. The widened semantics supplies a synthetic base state computed from the antichain of maximal common ancestors. No synthetic base version is allocated.

8.1 Resolver and widened merge

A resolver V contains

$$V.\mathsf{state} : \mathsf{Configuration}(D) \to \mathsf{Version} \to \mathsf{Version} \to \Sigma.$$

`StepV D V` includes every ordinary step through `base` and one additional merge constructor. For head states s_1, s_2, its new-version equation is

$$S'(v_m) = (\mathsf{merge}(V.\mathsf{state}(C, v_1, v_2), s_1, s_2), E_1 \cup E_2),$$

with the same head, parent, freshness, and rank updates as ordinary merge. It does not require a GCA.

Lean: `VirtualMergeBaseResolver, StepV, StepV.mergeVirtual`.

`IssuedStepV D V I` embeds an ordinary issued step or accepts a genuinely virtual `StepV` derivation that is not an ordinary `Step`. `MintCertifiedReachV D V I C` is its inductive closure from the initial configuration, retaining mint honesty before and after every step.

Lean: `IssuedStepV, MintCertifiedReachV`.

8.2 Recursive maximal-ancestor fold

The recursive fold needs a generalization of the pairwise definition. For a finite support X, let

$$\begin{aligned}\mathsf{CAWithAny}_P(X,w) &= \bigcup_{u\in X} \mathsf{CommonAncestors}(P,u,w),\\ \mathsf{IsMaximalWithAny}_P(X,w,m) &\iff m \in \mathsf{CAWithAny}_P(X,w)\\ &\quad \wedge \forall x \in \mathsf{CAWithAny}_P(X,w).\ m\ \mathsf{Reaches}\ x \to x = m.\end{aligned}$$

In other words, a candidate must be a common ancestor of w and at least one member of X; it need not be an ancestor of every member of X. Lean names these definitions `CommonAncestorsWithAny` and `IsMaximalCommonAncestorWithAny`. Write

$$\mathsf{BasesWithAny}(P,X,w) = \{m \mid \mathsf{IsMaximalWithAny}_P(X,w,m)\}$$

for the finite enumeration implemented by `maximalCommonAncestorsWithAny`. The singleton bridge is

$$\mathsf{IsMaximalWithAny}_P(\{a\},b,m) \iff \mathsf{IsMaximalCommonAncestor}(P,a,b,m).$$

Lean: `CommonAncestorsWithAny`, `IsMaximalCommonAncestorWithAny`, `isMaximalCommonAncestorWithAny_singleton`.

Lean: `maximalCommonAncestorsWithAny`, `mem_maximalCommonAncestorsWithAny_iff`.

Sort $\mathsf{BasesWithAny}(P,X,w)$ by increasing version rank. Let $\mathsf{stateD}(v)$ return the stored state, defaulting to σ_0 only to make the function total. Define the scratch fold vfold and its recursive base resolver vbase mutually:

$$\begin{aligned}\mathsf{vfold}(A,a,[]) &= a,\\ \mathsf{vfold}(A,a,m::ms) &= \mathsf{vfold}\big(A\cup\{m\}, \mathsf{merge}(\mathsf{vbase}(A,m),a,\mathsf{stateD}(m)), ms\big).\end{aligned}$$

The nested base is itself the fold of this sorted maximal antichain:

$$\mathsf{vbase}(X,w) = \begin{cases}\sigma_0, & \mathsf{sort}(\mathsf{BasesWithAny}(P,X,w)) = [],\\ \mathsf{vfold}(\{m_1\},\mathsf{stateD}(m_1),ms), & \mathsf{sort}(\mathsf{BasesWithAny}(P,X,w)) = m_1::ms.\end{cases}$$

Termination uses the cardinality of the joint ancestor support, lexicographically paired with pending-list length. The head-pair resolver is

$$\mathsf{virtualMergeBaseState}(C,v_1,v_2) = \mathsf{vbase}(\{v_1\},v_2).$$

The resolver whose state field is this function is $\mathsf{canonicalVirtualMergeBase}(D)$.

Lean: `vfoldAux`, `virtualBaseAux`, `virtualMergeBaseState`, `canonicalVirtualMergeBase`.

8.3 Virtual canonicity and adequacy

For finite $X \subseteq A_C$,

$$\mathsf{unionEvents}(C,X) = \bigcup_{v\in X} \mathsf{events}_C(v).$$

Under `StoreInv`, for allocated w, the maximal-base event union is exactly the outer intersection:

$$\mathsf{unionEvents}(C,\mathsf{BasesWithAny}(P,X,w)) = \mathsf{unionEvents}(C,X) \cap \mathsf{events}_C(w).$$

Lean: `unionEvents, maximalCommonAncestorsWithAny_unionEvents`.

The fold theorem states

$$\begin{aligned}&\mathsf{StoreInv}(S,P) \wedge \mathsf{CanonicalConfig}(C) \wedge \mathsf{JoinAt}(D,P,R_C)\\ &\wedge S(v_1) = (s_1,E_1) \wedge S(v_2) = (s_2,E_2)\\ &\Longrightarrow \mathsf{Canonical}(R_C, E_1 \cap E_2, \mathsf{virtualMergeBaseState}(C,v_1,v_2)).\end{aligned}$$

Lean: `virtualMergeBaseState_canonical`.

Consequently,

$$\begin{aligned}&\mathsf{Join}(D,P) \to \mathsf{Reachable}(\mathsf{StepV}, \mathsf{initConfig}(D), C) \to \mathsf{CanonicalConfig}(C),\\ &\mathsf{Join}(D,P) \to \mathsf{Reachable}(\mathsf{StepV}, \mathsf{initConfig}(D), C) \to \mathsf{HasReplayWitness}(C).\end{aligned}$$

Lean: `canonicalConfig_reachableV, replayWitnessV_of_join`.

As in the ordinary semantics, certified widened reachability erases to raw widened reachability. Hence the all-context Join theorem also supplies replay witnesses for certified widened executions.

Lean: `MintCertifiedReachV.toReachable`.

8.4 Verification certificates

A replay-adequacy certificate $\mathsf{ReplayAdequacyCertificate}(D,I,P)$ has field

$$\mathsf{soundV}: \quad \mathsf{MintCertifiedReachV}(D, \mathsf{canonicalVirtualMergeBase}(D), I, C) \to \mathsf{HasReplayWitness}(C).$$

Ordinary soundness follows because ordinary execution embeds in widened execution.

Lean: `ReplayAdequacyCertificate, ReplayAdequacyCertificate.sound`.

For a context predicate $G : \mathsf{ReplayContext}(D) \to \mathsf{Prop}$, define

$$\mathsf{IssuanceEstablishes}(D,I,G) \iff \forall C.\ \mathsf{MintHonest}(D, I.\mathsf{CanIssue}, C) \to G(R_C).$$

This is a proof obligation from mint honesty to G, not a theorem deriving mint honesty from local issuance checks.

Lean: `IssuanceEstablishes`.

The artifact provides two sufficient constructors:

$$\begin{aligned}\mathsf{Join}(D,P) &\Longrightarrow \mathsf{ReplayAdequacyCertificate}(D,I,P),\\ \mathsf{JoinOn}(D,P,G) \wedge \mathsf{IssuanceEstablishes}(D,I,G) &\Longrightarrow \mathsf{ReplayAdequacyCertificate}(D,I,P).\end{aligned}$$

The first constructor erases certified widened reachability before applying all-context Join; the second uses issuance consistency to discharge the explicit context condition.

Lean: `ReplayAdequacyCertificate.ofJoin`.

Lean: `ReplayAdequacyCertificate.ofJoinOn`.

The first accepts any issuance policy. For the second, certified reachability supplies mint honesty at each configuration; the separate proof of $\mathsf{IssuanceEstablishes}$ supplies $G(R_C)$, and $\mathsf{JoinOn}(D,P,G)$ supplies the required local merge theorem. For ordinary executions, the premises compose as

$$\begin{aligned}&\mathsf{JoinOn}(D,P,G) \wedge \mathsf{IssuanceEstablishes}(D,I,G)\\ &\wedge \mathsf{MintCertifiedReach}(D,I,C)\\ &\quad\Longrightarrow \mathsf{HasReplayWitness}(C).\end{aligned}$$

The corresponding widened theorem replaces the third premise with $\mathsf{MintCertifiedReachV}(D, \mathsf{canonicalVirtualMergeBase}(D), I, C)$.

Lean: `replayWitness_of_mintCertified, replayWitness_of_mintCertifiedV`.

These constructors do not exhaust the possible proofs: an instance may prove the replay-adequacy field directly by an execution invariant.

Let $\mathsf{CertifiedExecution}(D, I, C)$ mean either an ordinary $\mathsf{MintCertifiedReach}(D, I, C)$ derivation or a widened $\mathsf{MintCertifiedReachV}(D, \mathsf{canonicalVirtualMergeBase}(D), I, C)$ derivation. A sequential-correctness certificate has type

$$\forall C.\ \mathsf{CertifiedExecution}(D, I, C) \to \mathsf{HasReplayWitness}(C) \to \mathsf{IsSpecLinearizable}(D, P, \mathit{Spec}, \mathsf{Rel}, C).$$

Lean: `CertifiedExecution, SequentialCorrectnessCertificate`.

The complete package is

$$\begin{aligned}\mathsf{VerifiedMRDT}(D) = (&I : \mathsf{Issuance}(D),\\ &\mathit{ord} : \mathsf{EventOrder}(D),\\ &K : \mathsf{ReplayAdequacyCertificate}(D, I, \mathit{ord}),\\ &\mathit{Spec} : \mathsf{SequentialSpec}(D),\\ &\mathsf{Rel} : \Sigma \to \Sigma_S \to \mathsf{Prop},\\ &K_S : \mathsf{SequentialCorrectnessCertificate}(D, I, \mathit{ord}, \mathit{Spec}, \mathsf{Rel})).\end{aligned}$$

`PackagedMRDT` additionally contains a name and the signature D.

Lean: `VerifiedMRDT, PackagedMRDT`.

Its public conclusions are

$$\begin{aligned}\mathsf{MintCertifiedReach}(D, I, C) \to \mathsf{IsSpecLinearizable}(D, \mathit{ord}, \mathit{Spec}, \mathsf{Rel}, C),&\\ \mathsf{MintCertifiedReachV}(D, \mathsf{canonicalVirtualMergeBase}(D), I, C)&\\ \to \mathsf{IsSpecLinearizable}(D, \mathit{ord}, \mathit{Spec}, \mathsf{Rel}, C).&\end{aligned}$$

Lean: `VerifiedMRDT.correct, VerifiedMRDT.correctV`.

9 Distributed commit-history collection

Replicas fetch commits asynchronously, so each replica decides locally when history can be removed. It must retain enough authored evidence and enough graph information for future reachability and merge-base queries.

9.1 Local holdings and evidence

A local holding is

$$\mathsf{Local} = (\mathsf{head} : \mathsf{Version}, \mathsf{commits} : \mathcal{P}(\mathsf{Version}), \mathsf{headPresent} : \mathsf{head} \in \mathsf{commits}).$$

An envelope contains a commit set. Advertising exports the local set; receiving unions it into the destination without changing the head. A world is $\mathsf{World} = \mathsf{Replica} \to \mathsf{Local}$.

Lean: `GC.Local, GC.Envelope, GC.advertise, GC.receive, GC.World`.

Authorship has type $\mathsf{Author} = \mathsf{Version} \to \mathsf{Option}(\mathsf{Replica})$. Evidence for roster member r is

$$\mathsf{DerivedEvidence}(P, A, L, r) = \{v \mid v \in L.\mathsf{commits} \wedge A(v) = r \\ \wedge \mathsf{Reaches}(P, v, L.\mathsf{head})\}.$$

Evidence is complete for roster R at replica r_s when

$$\mathsf{EvidenceComplete}(P, A, R, r_s, L) \iff \forall r \in R.\ r = r_s \vee \\ \exists v.\ v \in \mathsf{DerivedEvidence}(P, A, L, r).$$

Lean: `GC.Author, GC.DerivedEvidence, GC.EvidenceComplete`.

9.2 Compression certificate

For any reachability relation R_V on versions, define

$$\mathsf{IsGCARel}(R_V, v_1, v_2, v_T) \iff R_V(v_T, v_1) \wedge R_V(v_T, v_2) \\ \wedge \forall w.\ R_V(w, v_1) \wedge R_V(w, v_2) \to R_V(w, v_T).$$

The canonical compressed edge summarizes any old path between retained endpoints:

$$\mathsf{compressedEdge}_{P,K}(a, b) \iff a \in K \wedge b \in K \wedge \mathsf{Reaches}(P, a, b).$$

Define

$$\mathsf{CompressedReaches}_{P,K} = \left(\mathsf{compressedEdge}_{P,K}\right)^*.$$

This relation preserves reachability exactly on retained nodes. Closure of K under maximal common ancestors suffices to preserve GCA answers; retaining intermediate paths or version 0 is unnecessary.

Lean: `GC.compressedEdge, GC.CompressedReaches, GC.compressedReaches_iff`.

Lean: `GC.compressed_isGCA_iff_of_maximalCommonAncestorClosed, GC.root_absent_when_dropped`.

A collection certificate chooses $K \subseteq L.\mathsf{commits}$ and proves:

1. evidence is complete, the local head is retained, and suitable evidence for every remote roster member is retained;
2. for $a, b \in K$, $\mathsf{CompressedReaches}_{P,K}(a, b)$ is equivalent to $\mathsf{Reaches}(P, a, b)$; and
3. for retained v_1, v_2, v_T, $\mathsf{IsGCARel}(\mathsf{CompressedReaches}_{P,K}, v_1, v_2, v_T)$ is equivalent to $\mathsf{IsGCA}(P, v_1, v_2, v_T)$.

Collection replaces the local commit set by K and preserves its head.

Lean: `GC.Certificate, GC.collect`.

Define the pairwise closure condition by

$$\mathsf{MaximalClosed}_P(K) \iff \forall a, b \in K.\ \forall m. \\ \mathsf{IsMaximalCommonAncestor}(P, a, b, m) \to m \in K.$$

The canonical constructor then has the schematic type

$$\begin{aligned} \mathsf{ofMaximalClosed} : {} & \mathsf{EvidenceComplete}(P, A, R, r_s, L) \to L.\mathsf{head} \in K \\ & \to (\forall r \in R.\ r = r_s \vee \exists v \in \mathsf{DerivedEvidence}(P, A, L, r).\ v \in K) \\ & \to K \subseteq L.\mathsf{commits} \to (\forall v, p.\ p \in P(v) \to p < v) \\ & \to \mathsf{MaximalClosed}_P(K) \to \mathsf{Certificate}. \end{aligned}$$

The exact Lean constructor is named in the source anchor.

Lean: `GC.Certificate.ofMaximalCommonAncestorClosed`.

9.3 Asynchronous refinement

The collecting protocol has fetch, head synchronization, and local collection. Its comparison protocol has fetch and head synchronization but no collection. Define

$$\begin{aligned}\mathsf{StoreSim}(F, C) &\iff F.\mathsf{head} = C.\mathsf{head} \\ &\quad \wedge C.\mathsf{commits} \subseteq F.\mathsf{commits} \wedge C.\mathsf{head} \in C.\mathsf{commits}, \\ \mathsf{WorldSim}(F, C) &\iff \forall r.\ \mathsf{StoreSim}(F(r), C(r)).\end{aligned}$$

One collecting step is matched by the corresponding network step or stuttering. Write Steps_{GC} and NoGCSteps for the reflexive-transitive closures of the collecting and comparison step relations, respectively. Then

$$\begin{aligned}&\mathsf{WorldSim}(F_0, C_0) \wedge \mathsf{Steps}_{GC}(C_0, C_1) \\ &\quad \Longrightarrow \exists F_1.\ \mathsf{NoGCSteps}(F_0, F_1) \wedge \mathsf{WorldSim}(F_1, C_1).\end{aligned}$$

Lean: `GC.StoreSim, GC.WorldSim, GC.execution_refines_noGC`.

10 Runtime and datatype-state collection

10.1 Commit-store runtime refinement

A datatype-rich runtime pairs semantics with local stores:

$$\mathsf{Runtime}(D) = (\mathsf{core} : \mathsf{Configuration}(D), \mathsf{stores} : \mathsf{World}).$$

It is well formed when each retained commit is allocated in the core and each semantic replica head agrees with its store head. A visible runtime transition contains an actual `Step` derivation and matching store evolution. Fetch and collection are silent.

Lean: `GC.Runtime, GC.Runtime.WellFormed, GC.RuntimeStep`.

Erasing optional labels removes silent steps:

$$\mathsf{eraseLabels} = \mathsf{List.filterMap}(\mathsf{id}).$$

Let RuntimeSteps be the list-labelled reflexive-transitive closure of runtime transitions, and let CoreSteps be the corresponding closure of raw `Step` transitions. Finite runtime traces refine raw semantic traces:

$$\mathsf{RuntimeSteps}(D, A, R, S_0, ls, S_1) \to \mathsf{CoreSteps}(D, S_0.\mathsf{core}, \mathsf{eraseLabels}(ls), S_1.\mathsf{core}).$$

The widened theorem uses the virtual-merge-base step relations and parameterizes availability by the exact versions read by the resolver.

Lean: `GC.eraseLabels, GC.runtime_refines_core, GC.runtime_refines_coreV`.

10.2 Datatype-state GC

A local collector is first specified independently of a runtime transition system. For an issuance policy I, a certificate supplies compact states C, collection evidence W, a relation c Rep s to

the full datatype state, an evidence predicate $\mathsf{EvidenceValid} : W \to C \to \Sigma \to \mathsf{Prop}$, a branch-compatibility predicate $\mathsf{Compatible} : \Sigma \to \Sigma \to \mathsf{Prop}$, and compact interpretations of initialization, update, ternary merge, and query. Its central obligations are

$$\begin{aligned} c\ \mathsf{Rep}\ s \wedge \mathsf{EvidenceValid}(w, c, s) &\Longrightarrow \mathsf{collect}(w, c)\ \mathsf{Rep}\ s, \\ c\ \mathsf{Rep}\ s \wedge I.\mathsf{CanIssue}(e, s) &\Longrightarrow \mathsf{update}_C(c, e)\ \mathsf{Rep}\ \mathsf{do}(s, e), \\ c_l\ \mathsf{Rep}\ l \wedge c_a\ \mathsf{Rep}\ a \wedge c_b\ \mathsf{Rep}\ b \wedge \mathsf{Compatible}(a, b) &\Longrightarrow \mathsf{merge}_C(c_l, c_a, c_b)\ \mathsf{Rep}\ \mathsf{merge}(l, a, b), \\ c\ \mathsf{Rep}\ s &\Longrightarrow \mathsf{query}_C(c, q) = \mathsf{query}(s, q). \end{aligned}$$

Together with the corresponding initialization law, this is `StateGCCertificate`. The generic `StateGCCertificate.exactState` chooses $C = \Sigma$, equality as Rep, and identity collection. It is a preservation baseline, not a reclamation result.

The more general `StateGCProtocol` embeds collection into physical execution. It supplies

$$\begin{aligned} \mathsf{Physical} &: \mathsf{Type}, \\ \mathsf{semantic} &: \mathsf{Physical} \to \mathsf{Configuration}(D), \\ \mathsf{Valid} &: \mathsf{Physical} \to \mathsf{Prop}, \\ \mathsf{PhysicalStep} &: \mathsf{Physical} \to \mathsf{Option}(\mathsf{Label}(D)) \to \mathsf{Physical} \to \mathsf{Prop}. \end{aligned}$$

It proves validity preservation, silent stuttering $\mathsf{semantic}(P') = \mathsf{semantic}(P)$, and visible refinement to `StepV D V`. Write $\mathsf{SemanticSteps}_V$ for the list-labelled reflexive-transitive closure of `StepV D V`. Every finite valid physical trace therefore erases to such a widened semantic trace.

Lean: `StateGCCertificate, StateGCCertificate.exactState, StateGCProtocol, StateGCProtocol.refines`.

10.3 Composition

The combined physical state pairs a datatype-state physical value with a commit world. A step is either a datatype-state step, with matching store evolution for a visible label, or a silent history fetch/collection step. Write Combined.Valid for componentwise validity and CombinedSteps for the list-labelled closure of these combined steps. `combinedProtocol` is itself a `StateGCProtocol`; hence

$$\begin{aligned} &\mathsf{Combined.Valid}(P) \wedge \mathsf{CombinedSteps}(P, ls, P') \\ &\quad \Longrightarrow \mathsf{SemanticSteps}_V(\mathsf{semantic}(P), \mathsf{eraseLabels}(ls), \mathsf{semantic}(P')). \end{aligned}$$

If the datatype protocol also proves that every visible physical step is raw, the ordinary refinement follows.

Lean: `GC.combinedProtocol, GC.CombinedSteps.refinesV, GC.CombinedSteps.refinesRaw`.

11 Verification certificates and instances

11.1 Formal instance profiles

In every profile, O is the datatype's operation type and $e = (t, r, o) \in \mathsf{Timestamp} \times \mathsf{Replica} \times O$ is an event: t is its timestamp, r its issuing replica, and o its operation payload. The operation forms below include metadata checked by issuance, not just the client's command. Each profile separates the datatype $D = (\Sigma, \mathsf{init}, O, Q, R, \mathsf{do}, \mathsf{merge}, \mathsf{query})$, its replay-order relation, issuance, and public sequential specification. Unless a profile states otherwise, $Q = \{\mathsf{rd}\}$ is the singleton read query (Lean's unit type); R is the stated query-result type. The sequential specification uses the

same query interface and supplies its own state, initial state, step, legality predicate, and query; the representation relation connects it to D. Global timestamp freshness and causal timestamp monotonicity come from the execution rules. Issuance checks whether a command may be created in its local state; *ord* specifies how conflicting events are ordered in replay. An identifier may occur in both rules without making either rule redundant.

Each profile below records the information needed to interpret the package: the representation, issuance predicate, sole *ord* relation, merge scope, public sequential state and legality predicate, representation relation, and proof route. The typed production registry contains 20 `PackagedMRDT` values; the profiles name the released entries belonging to each family. Queue, MVR, and the enriched FugueMax implementation are included. Helper lemmas remain in the cited instance modules. Initial states, observations, and abstract steps are specified below as well: these are part of the client contract, not consequences of the merge laws. Shared constructions are stated once and specialized in the relevant profile. Throughout these profiles, q denotes abstract state, q_0 its initial value, and $\mathsf{run}(u)$ the fold of the displayed abstract step from q_0 over a history prefix u. A prefix condition is required for every decomposition $\ell = u{+}{+}[e]{+}{+}v$. List filtering preserves order; set difference and list filtering do nothing to an absent item. The common certificate conclusion is the theorem stack in Section 8.4; additional datatype-specific conclusions are stated separately. Proof-local chain assignments and induction invariants are not additional client state.

Lean: `Production.registry, Production.names`.

The typed `Production.StateGC.registry` follows the same 20-entry order as the released-instance registry. Its four constructors distinguish an exact-state baseline, a representation-changing local certificate, an operational protocol, and a staged collector. Thus registry coverage means that the status of every released package is explicit; it does not mean that every package already reclaims state. For an exact-state baseline, the compact type is $C = \Sigma$, the relation is $\mathsf{Rep}(c, s) \iff c = s$, evidence is trivial, and $\mathsf{collect}((), c) = c$. It preserves all operations but reclaims nothing.

RGA, AegisSheet, TreeMove, and rich SidedPeritext currently package representation-changing proofs. RGA's entry is a lossless representation quotient, whereas the latter three reclaim datatype payload, hidden tree state, or rich-text metadata. The roadmap treats every remaining exact-state profile as an open classification obligation: supply a real collector when metadata is reclaimable, or prove that the released representation contains no semantically redundant internal state.

Lean: `StateGCCoverage, PackagedStateGC, Production.StateGC.registry, Production.StateGC.names`.

Grow-only stores. This profile covers the released grow-only-set, add-store, finite-add-store, flat-grow-only-set, and flat-grow-only-map entries.

Datatype D. For a store of α values, $O = \alpha$: the operation payload is the value to add, so an event is (t, r, x) with $x \in \alpha$. The grow-only set specializes α to $\mathbb{N}$. For AddStore and FinsetStore,

$$\Sigma = \mathcal{P}(\alpha) \text{ or } \mathsf{Finset}(\alpha), \qquad \mathsf{do}(s, e) = s \cup \{e.\mathsf{op}\}, \qquad \mathsf{merge}(l, a, b) = a \cup b.$$

Concrete state starts empty and its query is identity. The pointwise-Boolean grow-only set and map have operation type $O = A$; the payload is the key or key–value pair to add. They instead have state $A \to \mathsf{Bool}$, initially false, with

$$\mathsf{do}(s, e)(x) = s(x) \vee (x = e.\mathsf{op}), \qquad \mathsf{merge}(l, a, b)(x) = l(x) \vee a(x) \vee b(x).$$

Here A is the key type or key–value-pair type, respectively: the map stores pairs, not overwriting assignments. The ancestor is included in this raw Boolean merge, unlike the two-set union above. Its query is identity.

Replay-order relation ord. For every store variant, $\forall a, b.\ \neg(a \xrightarrow{\mathsf{ord}} b)$.

Issuance. For every variant, $\forall e, s.\ I.\mathsf{CanIssue}(e, s)$.

Sequential specification. For each variant, the abstract machine has the same state, initial value, update, and query as D. Explicitly, for each of the representations just defined,

$$\Sigma_{\mathrm{seq}} = \Sigma, \quad q_0 = \mathsf{init}, \quad \mathsf{step}(q, e) = \mathsf{do}(q, e),$$
$$\mathsf{query}_{\mathrm{seq}}(q, \mathsf{rd}) = q,$$
$$\mathsf{Legal}(\ell) \iff \mathsf{True}, \quad \mathsf{Rel}(s, q) \iff s = q.$$

Adding an already present value or pair leaves the state unchanged.

Lean: `AddStore.D, AddStore.generation, AddStore.spec, AddStore.verified`.

Lean: `FlatGrowOnly.D, FlatGrowOnly.spec, FlatGrowOnly.verified`.

Lean: `FinsetStore.spec`.

Datatype-state collection. Grow-only set, AddStore, FinsetStore, flat grow-only set, and flat grow-only map currently use the semantic set or pointwise map as the compact state:

$$C = \Sigma, \qquad c\ \mathsf{Rep}\ s \iff c = s, \qquad \mathsf{collect}((), c) = c.$$

Their registry entries therefore carry the exact-state certificate. Every stored member is part of the monotone semantic value under the current signature; a stronger result would have to prove a representation quotient or state formally that no internal metadata is reclaimable.

Counter. The single released ordinary Counter supports unrestricted increment and decrement.

Datatype D. Its operations are inc and dec. For the family parameterized by $\delta : O \to \mathbb{Z}$,

$$\Sigma = \mathbb{Z}, \qquad \mathsf{do}(s, e) = s + \delta(e.\mathsf{op}), \qquad \mathsf{merge}(l, a, b) = a + b - l.$$

Here $\delta(\mathsf{inc}) = 1$ and $\delta(\mathsf{dec}) = -1$. Concrete state starts at zero and its query is identity.

Replay-order relation ord. $\forall a, b.\ \neg(a \xrightarrow{\mathsf{ord}} b)$.

Issuance. $\forall e, s.\ I.\mathsf{CanIssue}(e, s)$.

Sequential specification. With the operation-specific δ above,

$$\Sigma_{\mathrm{seq}} = \mathbb{Z}, \quad q_0 = 0, \quad \mathsf{step}(q, (t, r, o)) = q + \delta(o), \quad \mathsf{query}_{\mathrm{seq}}(q, \mathsf{rd}) = q.$$

Here $\mathsf{Legal}(\ell) \iff \mathsf{True}$ and $\mathsf{Rel}(s, q) \iff s = q$. Counter decrement is unrestricted, including at zero; its result may be negative.

Lean: `FlatCounters.D, FlatCounters.generation, FlatCounters.spec, FlatCounters.verified`.

Datatype-state collection. The integer counter also carries an exact-state certificate. Its state is already an aggregate, not a replay log: one integer. Removing a component would need an explicit replica-retirement or coordinate-normalization protocol; none is part of the released semantics.

LWW register. *Datatype D.* The operation is $\mathsf{write}(v)$ for $v \in \mathbb{N}$. For a write event

$$e = (t, r, \mathsf{write}(v)), \qquad k(e) = (t, r, v),$$

order keys lexicographically by timestamp, then replica, then value. Distinct issued writes have distinct timestamps, so the remaining fields break ties only on arbitrary raw inputs. The state is either unset or a lexicographically ordered write,

$$\Sigma = \bot + \mathsf{Lex}(\mathsf{Timestamp}, \mathsf{Replica}, \mathbb{N}),$$
$$\mathsf{do}(s, e) = \max(s, k(e)), \qquad \mathsf{merge}(l, a, b) = \max(a, b).$$

The initial state is $\bot$. The query returns none at $\bot$ and the value component otherwise.

Replay-order relation ord.

$$e_1 \xrightarrow{\mathsf{ord}} e_2 \iff k(e_1) < k(e_2).$$

Issuance. $\forall e, s.\ I.\mathsf{CanIssue}(e, s)$.

Sequential specification.

$$\Sigma_{\mathrm{seq}} = \mathsf{Option}(\mathbb{N}), \qquad q_0 = \mathsf{none},$$
$$\mathsf{step}(q, (t, r, \mathsf{write}(v))) = \mathsf{some}(v), \qquad \mathsf{query}_{\mathrm{seq}}(q, \mathsf{rd}) = q,$$
$$\mathsf{Legal}(\ell) \iff \mathsf{True}, \qquad \mathsf{Rel}(s, q) \iff \mathsf{query}(s, \mathsf{rd}) = q.$$

The abstract step always overwrites, even for a lower-timestamp event; the witness order, not the step, supplies last-writer-wins resolution.

The sole *ord* policy orders unequal writes by their keys. The max updates commute, and the universal laws prove all-context Join under this same timestamp policy. The function `canonical` sorts the replay witness by key; `canonical_respects` proves that the result respects loOn. The theorem `replayOrder_order_acyclic` proves acyclicity by strict increase of the key. Chronological overwrite replay returns the stored maximal value, independently of concrete delivery order.

Lean: `LWWRegister.D, LWWRegister.replayOrder, LWWRegister.spec, LWWRegister.stateRel, LWWRegister.replayLaws, LWWRegister.join, LWWRegister.canonical_respects, LWWRegister.replayOrder_order_acyclic, LWWRegister.chronological_winner`.

Datatype-state collection. The exact-state baseline stores only the current winning tuple. Older writes are already absent from the semantic state, so this representation contains no retained replay log or tombstone metadata for a local collector to remove. Changing the tuple representation would be an encoding refinement rather than collection under the current interface.

Multi-value register. *Datatype D.* The operation is $\mathsf{write}(v, O)$, where $v \in \mathbb{N}$ is the new value and O is a list of overwritten timestamps, used through its membership predicate. Thus an event is $(t, r, \mathsf{write}(v, O))$; here the local payload name O denotes the overwrite list, not the operation type. The state is a finite set S of live tagged values, $\Sigma = \mathsf{Finset}(\mathsf{Timestamp} \times \mathbb{N})$, initially empty. Its query returns $\{v \mid \exists i.\ (i, v) \in S\}$. Writing $\mathsf{writeValue}((t, r, \mathsf{write}(v, O))) = v$,

$$\mathsf{do}(S, (t, r, \mathsf{write}(v, O))) = \{(i, w) \in S \mid i \notin O\} \cup \{(t, v)\},$$
$$\mathsf{merge}(L, A, B) = (L \cap A \cap B) \cup (A \setminus L) \cup (B \setminus L).$$

The ancestor distinguishes overwritten values from concurrent additions. Neither historical writes nor overwritten-tag tombstones are retained in S.

Replay-order relation ord.

$$a \xrightarrow{\mathsf{ord}} (t, r, \mathsf{write}(v, O)) \iff \mathsf{time}(a) \in O \qquad \text{for an issued write.}$$

The raw resolver additionally checks $\mathsf{time}(a) < t$ to exclude cyclic payloads on arbitrary inputs. For issued writes this inequality follows from issuance and is redundant, as proved by `replayOrder_iff_overwrite_of_execution`. Every edge between certified events follows visibility; concurrent writes do not choose a winner through *ord*.

Issuance. For $e = (t, r, \mathsf{write}(v, O))$, issuance requires

$$\begin{aligned}\mathsf{canIssue}(e, S) \iff &(\forall w.\ (t, w) \notin S) \\ &\wedge \forall i.\ \big(i \in O \iff \exists w.\ (i, w) \in S\big).\end{aligned}$$

Consequently an issued local write leaves exactly $\{(t, v)\}$. Replay removes only the carried targets, preserving unrelated concurrent writes.

Sequential specification. The public sequential state is a finite set q of live tagged values, initially empty, with

$$\mathsf{step}(q, (t, r, \mathsf{write}(v, O))) = \{(i, w) \in q \mid i \notin O\} \cup \{(t, v)\}.$$

Formally $\Sigma_{\text{seq}} = \mathsf{Finset}(\mathbb{N} \times \mathbb{N})$ and $q_0 = \varnothing$. Writing $\mathsf{times}(u)$ for the list of timestamps in u,

$$\begin{aligned}\mathsf{Legal}(\ell) &\iff \mathsf{Nodup}(\mathsf{times}(\ell)) \\ &\quad \wedge \forall u, t, r, v, O, w.\ \ell = u{+\!+}[(t, r, \mathsf{write}(v, O))]{+\!+}w \Longrightarrow O \subseteq \mathsf{times}(u), \\ \mathsf{query}_{\text{seq}}(q, \mathsf{rd}) &= \{v \mid \exists i.\ (i, v) \in q\}, \\ \mathsf{Rel}(S, q) &\iff S = q.\end{aligned}$$

Subset notation here tests membership, not list multiplicity. A target need not still be live at the replay prefix: concurrent writes may overwrite the same observed predecessor. Concrete and abstract states both contain only the surviving tagged values.

A chronological permutation supplies a legal loOn-respecting witness. For every stored version with event set E in the same ordinary or virtual certified execution, `mvr_correct` additionally proves

$$(i, v) \in S \iff \exists a \in E.\ \mathsf{time}(a) = i \wedge \mathsf{writeValue}(a) = v \wedge \forall b \in E.\ \neg(a\ \mathsf{vis}\ b).$$

Thus precisely the causally maximal writes survive, including concurrent maximal writes. Equal values from distinct writes appear once in the value-set query. On linear mint histories, `linear_register` proves that this query is empty initially and otherwise contains exactly the last written value. The proof maintains the surviving-write characterization for stored versions and recursively constructed virtual merge bases. It does not rely on all-commuting concrete updates or arbitrary-state Join.

Lean: `MVRLive.D, MVRLive.canIssue, MVRLive.replayOrder, MVRLive.spec, MVRLive.issued_write_singleton`.

Lean: `MVRLive.represented_of_mintCertifiedV`.

Lean: `MVRLive.verified, MVRLive.mvr_correct, MVRLive.linear_register, MVRLive.replayOrder_iff_overwrite_of_execution`.

Datatype-state collection. The registered representation already retains only live tagged values and has an exact-state baseline. Writes remove observed values during normal execution; no separate write-log collector is required. Historical commit versions and event payloads remain subject to their separate storage and history-collection obligations.

Bounded counter. Balegas et al., SRDS 2015, Section III-B, Figure 1, define a lower-bounded counter for a fixed set $I = \{0, \dots, n-1\}$ of replicas and bound $K \in \mathbb{Z}$. The state is $(R, U) \in \mathbb{N}^{I \times I} \times \mathbb{N}^I$, initially zero. The diagonal R_{ii} counts increments; R_{ij} for $i \neq j$ counts rights transferred from i to j; U_i counts decrements. Define

$$V(R,U) = K + \sum_{i \in I} R_{ii} - \sum_{i \in I} U_i, \qquad B_i(R,U) = R_{ii} + \sum_{j \in I \setminus \{i\}} R_{ji} - \sum_{j \in I \setminus \{i\}} R_{ij} - U_i.$$

Queries return V and B_i, respectively. For an event issued by $i \in I$ with amount $a \in \mathbb{N}$, the accepted updates are

$$\begin{array}{lll} \mathsf{do}((R,U), \mathsf{inc}(a), i) & = (R[(i,i) \mapsto R_{ii} + a], U), & \\ \mathsf{do}((R,U), \mathsf{dec}(a), i) & = (R, U[i \mapsto U_i + a]), & a \leq B_i(R,U), \\ \mathsf{do}((R,U), \mathsf{transfer}(a,j), i) & = (R[(i,j) \mapsto R_{ij} + a], U), & j \in I \setminus \{i\},\ a \leq B_i(R,U). \end{array}$$

Only issuer i writes row i and component U_i. Its updates are serialized. The conditions on the right are original-issuance conditions; the effect of an accepted operation is the displayed arithmetic update. Nonnegative amounts make the source's monotonicity assumption explicit; the distinct recipient follows its instruction to transfer to another replica. Merge is

$$\mathsf{merge}(l, (R,U), (R',U')) = \big((\max(R_{ij}, R'_{ij}))_{i,j}, (\max(U_i, U'_i))_i\big).$$

Thus the ancestor is unused. Different donors write different cells even when they transfer to the same recipient.

An independent sequential machine has a scalar $v \in \mathbb{Z}$ and balances $b : I \to \mathbb{Z}$, initially $(K, 0)$. Its transitions are

$$\begin{array}{lll} (v,b) \xrightarrow{\mathsf{inc}(a),i} (v + a, b[i \mapsto b_i + a]), & i \in I, & \\ (v,b) \xrightarrow{\mathsf{dec}(a),i} (v - a, b[i \mapsto b_i - a]), & i \in I, & a \leq b_i, \\ (v,b) \xrightarrow{\mathsf{transfer}(a,j),i} (v, b[i \mapsto b_i - a, j \mapsto b_j + a]), & i,j \in I,\ i \neq j, & a \leq b_i. \end{array}$$

The client value query returns v; the rights-accounting query returns b_i. The projection is $(R,U) \mapsto (V(R,U), B(R,U))$. Legal sequential runs preserve $v = K + \sum_i b_i$ and $b_i \geq 0$, hence $v \geq K$.

The five merge equations are checked with visibility as metadata dependence. For the causal-delta equation, same-replica visibility ensures that the issuing row in the remainder equals its row in the causal past; other rows dominate the past componentwise. The other four equations follow from maximum algebra. The checked sequential-history bridge transports original issuance funds to a causal replay prefix: that prefix contains every prior debit from the issuer, and additional events can only contribute incoming credits. Every certified stored version therefore has a legal explaining history and satisfies the local lower bound. The same argument applies to the finite union E of all issued events, whether or not any stored version contains this union:

$$K + \sum_{e \in E} \Delta(e) \geq K, \qquad \Delta(\mathsf{inc}(a)) = a, \quad \Delta(\mathsf{dec}(a)) = -a, \quad \Delta(\mathsf{transfer}(a,j)) = 0.$$

Finiteness of E is derived from execution. Both ordinary executions and executions with recursively reconstructed merge bases are covered. The public history, local-safety and global-bound theorem dependencies are audited to include all five VCs and the generic merge induction.

Lean: `BalegasBoundedCounter.localRights`, `BalegasBoundedCounter.value`, `BalegasBoundedCounter.applicable`.

Lean: `certifiedV, versions_safeV (BalegasBoundedCounterEvent)`.

Lean: `global_boundV (BalegasBoundedCounterEvent)`.

Datatype-state collection. No collector or replica-retirement protocol is supplied for this published counter.

Efficient observed-remove set. *Datatype D.* The operations are $\mathsf{add}(x)$ and $\mathsf{remove}(x)$ for $x \in \alpha$. An event is (t, r, o). Following Neem's efficient implementation, certified executions retain at most one record per replica and element. The raw state type is

$$\Sigma = \mathsf{Finset}(\mathsf{Replica} \times \mathsf{Timestamp} \times \alpha), \qquad s_0 = \varnothing.$$

$$\begin{aligned} \mathsf{do}(s, (t, r, \mathsf{add}(x))) &= \{(r, t, x)\} \cup \{(r', u, y) \in s \mid r' \neq r \vee y \neq x\}, \\ \mathsf{do}(s, (t, r, \mathsf{remove}(x))) &= \{(r', u, y) \in s \mid y \neq x\}, \\ \mathsf{merge}(l, a, b) &= (l \cap a \cap b) \cup (a \setminus l) \cup (b \setminus l). \end{aligned}$$

The query type is $Q = \alpha$ and the Boolean query at x tests whether $\exists r, t.\ (r, t, x) \in s$. There are no removed-tag tombstones.

Replay-order relation ord.

$$(t, r, \mathsf{remove}(x)) \xrightarrow{\mathsf{ord}} (u, r', \mathsf{add}(y)) \iff x = y,$$

and *ord* is false for all other operation pairs. This gives add-wins for concurrent add/remove pairs on the same element.

Issuance. $\mathsf{CanIssue}(e, s) = \mathsf{True}$. Fresh timestamps and sequential use of each replica are imposed by the execution semantics; no observed-tag payload is required.

Sequential specification. The state is the ordinary finite set q, with $q_0 = \varnothing$ and $\Sigma_{\mathrm{seq}} = \mathsf{Finset}(\alpha)$:

$$\begin{aligned} \mathsf{step}(q, (t, r, \mathsf{add}(x))) &= q \cup \{x\}, \\ \mathsf{step}(q, (t, r, \mathsf{remove}(x))) &= q \setminus \{x\}, \\ \mathsf{Legal}(\ell) &\iff \mathsf{True}, \qquad \mathsf{query}_{\mathrm{seq}}(q, x) = (x \in q). \end{aligned}$$

Both repeated addition and removal are idempotent. The representation relation is

$$\mathsf{Rel}(s, q) \iff q = \{x \mid \exists r, t.\ (r, t, x) \in s\}.$$

The public certificate covers ordinary and virtual-merge executions. Its execution invariant characterizes a stored record by its birth and the absence of a causally later removal or same-replica replacement. Three-way merge preserves this invariant; a sorted witness then reproduces the exact state while respecting IoOn. Same-replica additions need not commute as record updates, so the proof does not use the all-state replay-law route or add edges to the public *ord*.

Lean: `EfficientORSet.update, EfficientORSet.merge, EfficientORSet.replayOrder, EfficientORSet.issuance, EfficientORSet.spec, EfficientORSet.stateRel`.

Lean: `EfficientORSet.represented_of_mintCertifiedV, EfficientORSet.verified`.

Datatype-state collection. The efficient OR-set stores only live records, with at most one timestamp per replica/element pair; its registry entry is an exact-state baseline, not a separate collector certificate.

RGA. *Datatype D.* The operations are $\mathsf{addAfter}(a)$ and $\mathsf{remove}(i)$, where $a, i \in \mathbb{N}$ are the anchor and element identifiers; anchor zero denotes root. An insertion event is $(i, r, \mathsf{addAfter}(a))$: its timestamp is its new identifier i, not a separate payload field. The state is a pair $s = (s_1, s_2)$, where $s_1 : \mathbb{N} \times \mathbb{N} \to \mathsf{Bool}$ records insertions: $s_1(i, a) = \mathsf{true}$ means identifier i was inserted after anchor a. The second component $s_2 : \mathbb{N} \to \mathsf{Bool}$ records deletions: $s_2(i) = \mathsf{true}$ means identifier i has been removed. Both functions are initially everywhere false. Formally,

$$\begin{aligned}
\mathsf{do}(s, (i, r, \mathsf{addAfter}(a))) &= (\lambda(j, b).\ s_1(j, b) \vee (j = i \wedge b = a),\ s_2),\\
\mathsf{do}(s, (t, r, \mathsf{remove}(i))) &= (s_1,\ \lambda j.\ s_2(j) \vee j = i),\\
\mathsf{merge}(l, a, b)_k(x) &= l_k(x) \vee a_k(x) \vee b_k(x), \qquad k = 1, 2.
\end{aligned}$$

The argument x is an identifier–anchor pair for $k = 1$ and an identifier for $k = 2$. On finite supports, the concrete query sorts births by timestamp, folds root-prepend and after-anchor splices from the empty list, and then filters out graves; infinite Boolean supports are interpreted as empty supports by the total implementation.

Replay-order relation ord. Write $\mathsf{ins}(i, a)$ for an insertion with identifier i and anchor a, and $\mathsf{del}(x)$ for a deletion. The directed relation is

$$\begin{aligned}
\mathsf{ins}(i, a) \xrightarrow{\mathsf{ord}} \mathsf{ins}(j, b) &\iff (a = b \vee i = b) \wedge i < j,\\
\mathsf{ins}(i, a) \xrightarrow{\mathsf{ord}} \mathsf{del}(x) &\iff x = i \vee (a \neq 0 \wedge x = a).
\end{aligned}$$

Here $a = b$ identifies siblings, and $i = b$ means the second insertion uses the first as its anchor. It is false on the remaining directed forms. Thus *ord* orders sibling and dependent insertions by timestamp and puts a child insertion before a conflicting deletion of its identifier or non-root anchor.

Issuance. For the local state $s = (s_1, s_2)$, with insertion facts s_1 and deletion flags s_2 as above,

$$\begin{aligned}
&\mathsf{applicable}((i, r, \mathsf{addAfter}(a)), s)\\
&\quad\iff \big((a = 0 \wedge 0 < i) \vee (s_2(a) = \mathsf{false} \wedge a < i \wedge \exists p.\ s_1(a, p) = \mathsf{true})\big)\\
&\qquad \wedge (\forall a'.\ s_1(i, a') = \mathsf{false}) \wedge s_2(i) = \mathsf{false},\\
&\mathsf{applicable}((t, r, \mathsf{remove}(i)), s) \iff (\exists a.\ s_1(i, a) = \mathsf{true}) \wedge s_2(i) = \mathsf{false}.
\end{aligned}$$

Sequential specification. The abstract state is a list $q \in \mathsf{List}(\mathbb{N})$ of visible identifiers, initially $q_0 = []$. Its step is

$$\mathsf{step}(q, (i, r, \mathsf{addAfter}(a))) = \begin{cases} i :: q, & a = 0,\\ u +\!\!+ [a, i] +\!\!+ v, & a \neq 0,\ q = u +\!\!+ [a] +\!\!+ v,\ a \notin u,\\ q, & a \neq 0,\ a \notin q, \end{cases}$$

$$\mathsf{step}(q, (t, r, \mathsf{remove}(i))) = \mathsf{filter}(\lambda x.\ x \neq i, q).$$

Here $+\!\!+$ concatenates lists; the middle case inserts after the first occurrence of the anchor. Filtering preserves the order of all other identifiers. Deletion leaves no tombstone in the abstract state and is a no-op when i is absent, so repeated deletion is idempotent. The query and representation relation are

$$\mathsf{query}_{\mathrm{seq}}(q, \mathsf{rd}) = q, \qquad \mathsf{Rel}(s, q) \iff \mathsf{read}(s) = q.$$

$\mathsf{Legal}(\ell)$ holds exactly when its timestamps are unique and, for every decomposition $\ell = u +\!\!+ [e] +\!\!+ v$,

$$\begin{cases} 0 < i\ \wedge \big(a = 0\ \vee\ \exists r', p.\ (a, r', \mathsf{addAfter}(p)) \in u\big), & e = (i, r, \mathsf{addAfter}(a)),\\ \exists r', a.\ (i, r', \mathsf{addAfter}(a)) \in u, & e = (t, r, \mathsf{remove}(i)). \end{cases}$$

Thus legality requires an earlier insertion of each non-root anchor or deletion target, not its continued presence in the abstract list. Repeated deletions remain legal; local issuance still requires the target to be live. The implementation retains birth and grave facts internally, including for integration of operations minted concurrently with deletion. Issuance rejects a new insertion after an observed anchor deletion. RGA proves all-context Join through the universal laws; issuance establishes well-formedness of the ordinary-list witness.

Lean: `RGA.applicable, RGA.generation, RGA.join`.

Lean: `RGA.listStep, RGA.listLegal, RGA.listSpec, RGA.replayOrder, RGA.listRel, RGA.replayOrder_insert_iff_original_of_execution, RGA.canonical_respects_replayOrder, RGA.verified`.

Datatype-state collection. RGA has a checked representation compactor, but not deleted-identifier reclamation. Its finite state consists of birth pairs (i, p) and a graveyard G. Honest issuance establishes that every grave has a birth. The alternate representation stores the same birth pairs and the live identifiers:

$$C = \{\mathsf{parents} : \mathsf{Finset}(\mathbb{N} \times \mathbb{N}), \quad \mathsf{live} : \mathsf{Finset}(\mathbb{N})\}, \qquad \mathsf{live} = \mathsf{birthIds} \setminus G.$$

Materialization retains the birth pairs and restores $G = \mathsf{parentIds} \setminus \mathsf{live}$. The theorem `unpack_pack` proves that this round trip is exact on well-formed states. The packaged certificate proves preservation by collection, guarded update, ternary merge, and the ordered list query. With the explicit cost model of two words per birth plus one per grave or live identifier, this conversion is smaller when the live set is smaller than the graveyard. It is not an unconditional size reduction.

This result deliberately retains one parent fact for every deleted identifier. Issuance rejects insertion after an observed deleted anchor. The checked controls establish rejection both before and after erasing that anchor; they do not establish an impossibility of reclamation. A collector that removes birth pairs would need a separate continuation proof covering operations issued concurrently with deletion.

Lean: `RGA.GC.pack, RGA.GC.unpack, RGA.GC.packedWords_pack_lt_of_live_lt_grave, RGA.GC.certificate, RGA.GC.future_after_dead_anchor_not_applicable, RGA.GC.erase_dead_anchor_rejects_future_issuance`.

TreeMove. *Datatype D.* The operation is a move record $(\mathit{parent}, \mathit{metadata}, \mathit{child}) \in \mathbb{N}^3$. An event pairs this payload with its timestamp t and issuing replica r. The state is a finite set of timestamped moves. Rendering sorts the set and applies each move to an ordinary tree. Initially the event set is empty; update inserts the event and merge unions the two branches. The sorting key is the lexicographic tuple $(t, r, \mathit{parent}, \mathit{metadata}, \mathit{child})$. A tree is a partial map from child to parent–metadata pairs, initially everywhere none. A step replaces the named child's binding if the resulting child-to-parent graph is acyclic, and otherwise leaves the tree unchanged. Root is zero and trash is one. The query hides trash and every node with a nonempty parent path to trash, returning the remaining partial map. Rendering folds these guarded steps in key order.

Replay-order relation ord. The resolver is all-Either, hence $\forall a, b.\ \neg(a \xrightarrow{\mathsf{ord}} b)$.

Issuance.

$$\begin{aligned}\mathsf{applicable}(e, s) \iff\ & \mathsf{freshKey}_s(e) \wedge \mathsf{visibleNode}_s(e.\mathsf{parent}) \\ & \wedge (\neg\mathsf{knownNode}_s(e.\mathsf{child}) \vee \mathsf{visibleNode}_s(e.\mathsf{child})) \\ & \wedge e.\mathsf{child} \neq \mathsf{root} \wedge e.\mathsf{child} \neq \mathsf{trash}.\end{aligned}$$

Here freshness excludes a stored event with the same timestamp–replica pair. A node is known if it is root, trash, or a stored move's child; it is visible if it is known and neither trash nor below trash in the rendered tree.

Sequential specification. The state is $q : \mathbb{N} \to \mathsf{Option}(\mathbb{N} \times \mathbb{N})$, with $q_0(x) = \mathsf{none}$. Write $x \to_q y$ when $q(x) = \mathsf{some}(y, m)$ for some metadata m. For a move event $e = (t, r, (p, m, c))$, put $q' = q[c \mapsto \mathsf{some}(p, m)]$. Then

$$\mathsf{step}(q, e) = \begin{cases} q', & \mathsf{Acyclic}(\to_{q'}), \\ q, & \text{otherwise}, \end{cases}$$

$$\mathsf{query}_{\mathrm{seq}}(q, \mathsf{rd})(x) = \begin{cases} \mathsf{none}, & x = 1 \lor x \to_q^+ 1, \\ q(x), & \text{otherwise}, \end{cases}$$

$$\begin{aligned} \mathsf{Legal}(\pi) \iff{}& \mathsf{Nodup}(\pi) \land \mathsf{Pairwise}(\mathsf{eventLE}, \pi) \\ & \land \forall u, e, v.\ \pi = u{+}{+}[e]{+}{+}v \implies \\ & \quad \exists E_0 \subseteq \mathsf{toFinset}(u).\ \mathsf{applicable}(e, E_0), \\ \mathsf{Rel}(E, q) \iff{}& q = \mathsf{run}(\mathsf{sort}_{\mathsf{eventLE}}(E)). \end{aligned}$$

Here $\mathsf{eventLE}(a, b)$ compares the full keys displayed above, and $\to_q^+$ is a nonempty parent path. A move can allocate an absent child; there is no separate delete operation. Moving a subtree to trash hides it without erasing its bindings. A cyclic move is a no-op, not an illegal history step. Each move must also have an issuable origin within the replay prefix: this retains destination visibility and reserved-node restrictions in the public contract. The origin need not be the entire prefix, which may include unrelated concurrent moves. Certified execution supplies the actual causal origin; sorting places all its events before the move. Join is all-context by the universal-law route; the package also contains a separate tree-safety certificate.

Lean: `TreeMove.applicable, TreeMove.ClientLegal, TreeMove.clientLegal_origin, TreeMove.sequentialSpec, TreeMove.verified`.

Datatype-state collection. TreeMove has an operational collector with

$$C = \{\mathsf{base} : \mathsf{Tree},\ \mathsf{pending} : \mathsf{List}(\mathsf{Event})\}, \qquad \mathsf{materialize}(c) = \mathsf{replayFrom}(c.\mathsf{base}, c.\mathsf{pending}).$$

A `StableCut` splits canonical replay into a stable prefix and pending suffix: $\mathsf{orderedEvents}(E) = u{+}{+}v$. Collection returns $(\mathsf{replayFrom}(\mathsf{emptyTree}, u), v)$ and preserves the exact materialized tree. The trash collector returns $(\mathsf{visibleTree}(c.\mathsf{base}), c.\mathsf{pending})$; its query preservation theorem requires $c.\mathsf{pending} = []$. The representation relation equates the compact visible tree with the query of the full event set, and the packaged protocol proves silent collection plus visible ordinary and widened execution.

Lean: `TreeMove.GC.StableCut, TreeMove.GC.Compact, TreeMove.GC.collectPrefix, TreeMove.GC.collectTrash, TreeMove.GC.collectTrash_query_of_empty_suffix, TreeMove.GC.protocol`.

AegisSheet. *Datatype D.* An operation carries an observed timestamp set and a command, either $\mathsf{direct}(a)$ or $\mathsf{undo}(u, a)$, where u names the event to undo and a is the carried action. Actions are axis updates (axis, identifier, insert/move/remove/restore kind, optional before/after position), cell updates (row, column, before/after value sets, overwritten timestamps), range updates (identifier, optional before/after four-anchor range, overwritten timestamps), or purges (cutoff, coordinates, covered timestamp–coordinate pairs, acknowledging replicas). An event adds its timestamp

and issuing replica to this payload. The MRDT stores a finite set of these events and merges by union. It starts empty and update inserts the event. The query is the sheet view defined with the sequential specification below.

Replay-order relation ord. Its resolver is all-Either, hence $\forall a, b.\ \neg(a \xrightarrow{\mathsf{ord}} b)$.

Issuance.

$$\mathsf{applicable}(e, E) \iff \mathsf{applicableB}(e, E) = \mathsf{true} \wedge \forall t \in \mathsf{eventTimes}(E).\ t < e.\mathsf{time}.$$

The Boolean part checks exact causal-origin metadata, action metadata, direct-action preconditions, and undo provenance. The guard requires no separate freshness test: the strict clock condition implies that the new timestamp is absent from the origin's timestamp set. The guard requires the carried observed set to equal all origin timestamps, including timestamps recorded in purge coverage. Overwrite targets must be earlier events at the same cell or range. Axis insertion names a new identifier; move and removal match the current position, with respectively a present or absent after-position. Direct restore is disallowed. Cell updates require live row and column identifiers, the exact current value set, and exact active-version timestamps; range updates require the analogous exact range values and active timestamps. A purge requires issuer zero, acknowledgements from replicas zero, one, and two, covered writes at or below the cutoff at named coordinates, and a nonlive axis at every purged coordinate. Undo names an observed event by the same issuer, excludes purge, and carries exactly its inverse: swap before/after, use restore for an axis, and name the original timestamp as the overwritten version for a cell or range.

Sequential specification. Write the state as $q = (K_R, K_C, T_R, T_C, P_R, P_C, V, H, M)$, with components

$$\begin{aligned} K_X &\in \mathsf{Finset}(\mathbb{N}), \qquad T_X : \mathbb{N} \to \mathsf{Finset}(\mathsf{Timestamp}), \\ P_X &: \mathbb{N} \to \mathsf{Finset}(\mathsf{Timestamp} \times \mathsf{Position}), \\ V &: \mathbb{N}^2 \to \mathsf{Finset}(\mathsf{Timestamp} \times \mathbb{N}), \\ H &: \mathbb{N} \to \mathsf{Finset}(\mathsf{Timestamp} \times \mathsf{RangeSpec}), \qquad M \in \mathsf{Finset}(\mathsf{Purge}). \end{aligned}$$

Here $X \in \{R, C\}$ selects rows or columns: K_X contains known identifiers, T_X their keep tokens, and P_X their position versions. V holds cell versions, H range versions, and M purge markers. A position is a natural number; a range specification has four stable row/column anchors. Every set and every function value in q_0 is empty.

For e with timestamp t and observed set S, the following assignments define $\mathsf{step}(q, e)$; unmentioned components are unchanged. They interpret the carried action for both direct and undo commands.

- Axis X, identifier i, after-position b:

$$K'_X = K_X \cup \{i\}, \qquad (T'_X(i), P'_X(i)) = \begin{cases} (T_X(i) \cup \{t\}, P_X(i) \cup \{(t, p)\}), & b = \mathsf{some}(p), \\ (T_X(i) \setminus S, P_X(i)), & b = \mathsf{none}. \end{cases}$$

- Cell (i, j), overwritten timestamps O, after-values A:

$$V'(i,j) = \{(u, v) \in V(i,j) \mid u \notin O\} \cup (\{t\} \times A),$$
$$K'_R = K_R \cup \{i\}, \quad K'_C = K_C \cup \{j\}, \quad T'_R(i) = T_R(i) \cup \{t\}, \quad T'_C(j) = T_C(j) \cup \{t\}.$$

- Range i, overwritten timestamps O, optional after-specification b:

$$H'(i) = \{(u, h) \in H(i) \mid u \notin O\} \cup \begin{cases} \{(t, h)\}, & b = \mathsf{some}(h), \\ \varnothing, & b = \mathsf{none}. \end{cases}$$

- Purge marker m:

$$M' = M \cup \{m\}, \qquad V'(i,j) = \{(u,v) \in V(i,j) \mid (u,(i,j)) \notin m.\mathsf{covered}\} \quad \text{for every } i,j.$$

Absent overwrite targets are harmless. Axis removal removes observed keep tokens, not positions or cell versions. Empty cell after-values still add axis keep tokens; range removal adds no new version. Undo uses these same steps on its carried inverse action, rather than rolling back the whole state.

Define $\mathsf{versionPurged}(q,u,i,j)$ to mean $\exists m \in M.\ (i,j) \in m.\mathsf{coordinates} \wedge u \le m.\mathsf{cutoff}$. The query $\mathsf{query}_{\mathrm{seq}}(q,\mathsf{rd})$ is the view with

$$\begin{aligned}
\mathsf{axes}_X(q) &= \{i \in K_X \mid T_X(i) \neq \varnothing\},\\
\mathsf{position}_X(q,i) &= \{p \mid \exists u.\ (u,p) \in P_X(i) \wedge \forall (v,p') \in P_X(i).\ v \le u\},\\
\mathsf{cell}(q,i,j) &= \begin{cases} \{v \mid \exists u.\ (u,v) \in V(i,j) \wedge \neg\mathsf{versionPurged}(q,u,i,j)\}, & T_R(i) \neq \varnothing \wedge T_C(j) \neq \varnothing,\\ \varnothing, & \text{otherwise,} \end{cases}\\
\mathsf{range}(q,i) &= \{h \mid \exists u.\ (u,h) \in H(i)\}.
\end{aligned}$$

An empty position set yields no position; tied maximal timestamps yield all positions at that timestamp. The cutoff mask is part of this contract, not merely a storage optimization. The public legality predicate is

$$\begin{aligned}
\mathsf{CausalOriginLegal}(\pi) \iff\ & \mathsf{Chronological}(\pi) \wedge\\
& \forall \pi_0, e, \pi_1.\ \pi = \pi_0 +\!\!+ e :: \pi_1 \to \exists E_0 \subseteq \mathsf{toFinset}(\pi_0).\ \mathsf{applicable}(e, E_0).
\end{aligned}$$

Here chronological means strictly increasing event timestamps. Event-set materialization collects the axis facts and timestamped values, subtracts observed axis-removal tokens and overwritten or covered versions, and retains purges. It is defined without choosing an enumeration. Writing $\mathsf{materialize}(E)$ for that event-set construction, the remaining components are

$$\begin{aligned}
\mathsf{Legal}(\ell) &\iff \mathsf{CausalOriginLegal}(\ell),\\
\mathsf{Rel}(E,q) &\iff (\forall \ell.\ \mathsf{Chronological}(\ell) \wedge \mathsf{toFinset}(\ell) = E \implies \mathsf{run}(\ell) = q)\\
&\qquad \wedge q = \mathsf{materialize}(E) \wedge \mathsf{query}_{\mathrm{seq}}(q,\mathsf{rd}) = \mathsf{query}(E,\mathsf{rd}).
\end{aligned}$$

Join is all-context by universal laws; issuance establishes causal-origin legality and the safety invariants.

Lean: `AegisSheet.applicable`, `AegisSheet.ClockedAt.fresh`, `AegisSheet.applicable_iff_explicit_freshness`, `AegisSheet.generation`, `AegisSheet.join`.

Lean: `AegisSheet.Sequential.CausalOriginLegal`, `AegisSheet.Sequential.step`, `AegisSheet.Sequential.view`, `AegisSheet.Sequential.materializedStateRel`, `AegisSheet.Sequential.clientSpec`, `AegisSheet.verified`.

Datatype-state collection. The current AegisSheet MRDT stores an event set and merges it by union. Its collector retains replicated purge markers but removes exactly the covered cell payloads:

$$\mathsf{semanticCollect}(E) = \{e \in E \mid e \text{ is not a cell update covered by a retained marker}\}.$$

The compact state is another finite event set, and

$$c\ \mathsf{Rep}\ E \iff c = \mathsf{semanticCollect}(E) \wedge \mathsf{StateValid}(E) \wedge \mathsf{TimestampUnique}(E).$$

The certificate normalizes after collection, update, and union merge and proves equality of the whole sheet view. This is genuine payload reclamation, but it is still a collector for the event-set encoding; it is not yet the proposed materialized three-way AegisSheet representation.

Lean: `AegisSheet.GC.semanticCollect`, `AegisSheet.GC.Represents`, `AegisSheet.GC.certificate`.

EmbedRGA and Peritext. *Datatype D.* EmbedRGA has operations $\mathsf{ins}(x, p, a)$ and $\mathsf{del}(i)$, where $x \in \alpha$ is the inserted payload, p a list of Boolean coordinate bits, and $a, i \in \mathbb{N}$ the anchor and deletion target. An insertion's identifier is its event timestamp. Fix an ordered prefix code Γ for positive natural numbers. Concrete state is a list of $(id, payload, coordinate)$ records, initially empty. Insertion at timestamp t constructs coordinate $p{+}{+}\Gamma.\mathsf{enc}(t - a)$ and inserts the record in descending lexicographic key order, unless its identifier is already present. The key maps coordinate bits false/true to 1/2 and appends the terminator 3. Deletion filters the named identifier; the query projects payloads. For ancestor L and branches A, B, merge keeps from A records whose identifiers are in B or absent from L, and from B records whose identifiers are absent from both L and A. It merges these two filtered lists in key order (choosing the second list at a key tie).

Replay-order relation ord.

$$\mathsf{ins}^{t}_{x,p,a} \xrightarrow{\mathsf{ord}} \mathsf{del}_i \iff t = i,$$

and is false on all other directed forms. In particular, immutable coordinates make insertion/insertion pairs commute without an *ord* edge.

Issuance.

$$\mathsf{eApplicable}(e, s) \iff \begin{cases} a < e.\mathsf{time} \wedge ((a = 0 \wedge p = []) \vee \exists x.\ (a, x, p) \in s), & e.\mathsf{op} = \mathsf{ins}(x, p, a), \\ i \in \mathsf{ids}(s), & e.\mathsf{op} = \mathsf{del}(i). \end{cases}$$

$\mathsf{EHonestCore}(R)$ requires each deletion target to have a visible insertion. It also requires one assignment $i \mapsto c_i$ of finite lists of positive natural numbers such that each insertion of identifier i has coordinate $\mathsf{coordOf}(c_i)$ and $\sum c_i = i$. Here $\mathsf{coordOf}$ concatenates the prefix codes of the list entries. The direct theorem is $\mathsf{JoinOn}(E, \mathsf{EHonestCore})$, and mint honesty for `eApplicable` establishes that condition.

Sequential specification. The state is $\Sigma_{\mathrm{seq}} = \mathsf{List}(\mathbb{N} \times \alpha)$, with $q_0 = []$. The abstract insertion ignores the carried coordinate p:

$$\mathsf{step}(q, (t, r, \mathsf{ins}(x, p, a))) = \begin{cases} (t, x) :: q, & a = 0, \\ u{+}{+}[(a, y), (t, x)]{+}{+}v, & a \neq 0,\ q = u{+}{+}[(a, y)]{+}{+}v, \\ q, & a \neq 0,\ a \notin \mathsf{ids}(q), \end{cases}$$

$$\mathsf{step}(q, (t, r, \mathsf{del}(i))) = \mathsf{filter}(\lambda z.\ z.\mathsf{id} \neq i, q),$$

$$\mathsf{query}_{\mathrm{seq}}(q, \mathsf{rd}) = \mathsf{map}(\mathsf{payload}, q),$$

$$\mathsf{Rel}(s, q) \iff \mathsf{map}(\mathsf{identifierPayload}, s) = q.$$

The split uses the first occurrence of a; an absent anchor makes insertion a no-op, and an absent target makes deletion a no-op. For each prefix u, let $B(u)$ be its insertion events and $c(b)$ the coordinate computed from insertion b by the concrete coordinate formula above. Then $\mathsf{Legal}(\ell)$ holds exactly when every prefix decomposition satisfies

$$\begin{cases} (\forall b \in B(u).\ \mathsf{time}(b) < t) \wedge \\ \quad \big((a = 0 \wedge p = []) \vee \exists b \in B(u).\ \mathsf{time}(b) = a \wedge c(b) = p\big), & e = (t, r, \mathsf{ins}(x, p, a)), \\ \exists b \in B(u).\ \mathsf{time}(b) = i, & e = (t, r, \mathsf{del}(i)). \end{cases}$$

This is earlier allocation, not continued liveness: repeated deletion remains legal. Immutable coordinates remain implementation and event metadata. Peritext instantiates the payload with characters and mark boundaries and inherits this certificate; its additional rendering theorem maps the certified buffer to marked text. The Peritext payload is either a character codepoint or $\mathsf{bound}(i, m, b)$, where b selects start/end and m is bold, italic, underline, strike, link(URL), comment(author,text), color(RGB), or heading(level). Its public query is the payload list, including boundaries. Rendering is a separate fold with an initially empty list of open (i, m) marks: a start prepends its mark, an end removes all marks with that identifier, and a character emits its codepoint together with the current open marks.

Lean: `EmbedRGA.EHonestCore, EmbedRGA.eApplicable, EmbedRGA.e_join_at`.

Lean: `ProductionRGA.embedLegal, ProductionRGA.embedClientSpec, ProductionRGA.embedRel, ProductionRGA.embed`.

Lean: `Peritext.verified`.

Lean: `EmbedRGA.eSpecStep`.

Lean: `Peritext.render`.

Datatype-state collection. EmbedRGA, its Peritext specialization, and their current live-sequence representations use exact-state certificates. Deletion is represented by removal from the live sequence rather than a separate graveyard, but surviving coordinates and anchors may still constrain later insertions. A smaller physical encoding therefore needs a continuation theorem for coordinate and anchor summaries, not merely equality of the current rendered list.

SidedEmbedRGA and SidedPeritext. *Datatype D.* SidedEmbedRGA adds a left/right choice d to insertion, with natural-number payloads: its operations are $\mathsf{ins}(x, p, a, d)$ and $\mathsf{del}(i)$, where $x, a, i \in \mathbb{N}$, p is a natural-number coordinate list, and $d \in \{\mathsf{L}, \mathsf{R}\}$. Concrete state, deletion, merge, and payload query follow the preceding profile, but coordinates are natural-number lists with key $c{+}{+}[3]$. An insertion's coordinate is p followed by the code of $t - a$: for right insertion map false/true to 1/2; for left insertion complement the bits and map false/true to 4/5.

Replay-order relation ord.

$$\mathsf{ins}^{t}_{x,p,a,d} \xrightarrow{\mathsf{ord}} \mathsf{del}_i \iff t = i,$$

with no other directed edges; the side d is already part of the immutable coordinate.

Issuance. The issuer requires precisely the same $a < t$ and root-empty-prefix or live-anchor-coordinate condition as EmbedRGA; deletion requires a live identifier. The side does not strengthen this guard.

Sequential specification. The state is $\Sigma_{\mathrm{seq}} = \mathsf{List}(\mathbb{N} \times \mathbb{N})$, with $q_0 = [(0, 0)]$. For the first occurrence $q = u{+}{+}[(a, y)]{+}{+}v$ of the parent,

$$\begin{aligned}
\mathsf{step}(q, (t, r, \mathsf{ins}(x, p, a, \mathsf{L}))) &= u{+}{+}[(t, x), (a, y)]{+}{+}v,\\
\mathsf{step}(q, (t, r, \mathsf{ins}(x, p, a, \mathsf{R}))) &= u{+}{+}[(a, y), (t, x)]{+}{+}v,\\
\mathsf{step}(q, (t, r, \mathsf{del}(i))) &= \mathsf{filter}(\lambda z.\ z.\mathsf{id} \neq i, q).
\end{aligned}$$

Both insertion cases return q when the parent is absent, including root if it is absent. Deletion of an absent target is a no-op. The query is

$$\mathsf{query}_{\mathrm{seq}}(q, \mathsf{rd}) = \mathsf{map}(\mathsf{payload}, \mathsf{filter}(\lambda z.\ z.\mathsf{id} \neq 0, q)).$$

Its relation is

$$\mathsf{Rel}(s, q) \iff \mathsf{map}(\mathsf{identifierPayload}, s) = \mathsf{filter}(\lambda p.\ p.\mathsf{id} \neq 0, q).$$

Legal is the prefix formula in the preceding profile, with $\mathsf{ins}(x, p, a, d)$ in place of $\mathsf{ins}(x, p, a)$ and $c(b)$ computed using the sided coordinate formula. Thus side changes the splice and coordinate, not the allocation requirement. For the merge proof, $\mathsf{SHonestCore}(R)$ requires each deletion to have a visible insertion of its target and one assignment of finite sided chains $c_i = [(d_1, \delta_1), \ldots, (d_n, \delta_n)]$ to insertion identifiers. Each δ_k is positive, $\sum_k \delta_k = i$, and the insertion coordinate is the concatenation of their side-specific codes defined above. Its merge theorem is $\mathsf{JoinOn}(S, \mathsf{SHonestCore})$.

Lean: `SidedEmbedRGA.SHonestCore, SidedEmbedRGA.sApplicable, SidedEmbedRGA.s_join_at`.

Lean: `SidedEmbedRGA.sSpecStep`.

Lean: `ProductionRGA.sidedLegal, ProductionRGA.sidedClientSpec, ProductionRGA.sidedRel`.

Datatype-state collection. The sided sequence uses the exact-state baseline on its live coordinate list. No representation-changing collector is provided.

SidedPeritext variants.

Datatype D. SidedPeritext composes the sided sequence with grow-only deletion and mark stores. Its operations inject one sided text operation, one deletion-store identifier, or one mark record into the respective component; these are written $\mathsf{textIns}(x, p, a, d)$, $\mathsf{textDel}(i)$, $\mathsf{deleteStoreAdd}(i)$, and $\mathsf{markStoreAdd}(m)$ below. Both stores start empty, add their supplied items, and merge by union. The sequence component uses the sided datatype above. Core queries select text, deletion store, or mark store, so Q is a three-way sum of singleton queries. The rich query instead takes a formatting kind (bold, italic, underline, strike, link, comment, color, or heading) and returns a list of codepoint–Boolean pairs, as defined below.

Replay-order relation ord. Its *ord* lifts the SidedEmbedRGA relation on two text events and returns Either for store/store and text/store pairs. Formally, if ι injects a text event into the product,

$$\iota(e_1) \xrightarrow{\mathsf{ord}} \iota(e_2) \iff e_1 \xrightarrow{\mathsf{ord}} e_2, \qquad \neg(s_1 \xrightarrow{\mathsf{ord}} s_2), \quad \neg(\iota(e) \xrightarrow{\mathsf{ord}} s), \quad \neg(s \xrightarrow{\mathsf{ord}} \iota(e))$$

for store events s, s_1, s_2. The rich package changes only the query boundary to rendered marks, not the sequential machine or *ord*.

Issuance. For an event e and state (s_T, s_D, s_M), its cross-component guard is

$$\mathsf{coreGuard}(e, s_T, s_D, s_M) \iff \begin{cases} \mathsf{sApplicable}(e, s_T), & e = \mathsf{textIns}, \\ \mathsf{False}, & e = \mathsf{textDel}, \\ i \in \mathsf{ids}(s_T) \wedge i \notin s_D, & e = \mathsf{deleteStoreAdd}(i), \\ \mathsf{validMark}(e, s_T, s_D, s_M), & e = \mathsf{markStoreAdd}. \end{cases}$$

Here a mark contains its identifier, formatting kind, add/remove flag, value, two anchor identifiers, and each boundary's before/after gravity. validMark means its identifier equals the event timestamp, both anchors occur in s_T and neither is in s_D, and no stored mark has that identifier. Recorded ancestry fields are not used by the document-order query. The merge theorem composes projected SHonestCore with the two all-context grow-only-store theorems.

Sequential specification. Write $q = (q_T, q_D, q_M)$ for an ordinary identifier–value list, a finite deletion set, and a finite mark set, respectively; $q_0 = ([(0, 0)], \varnothing, \varnothing)$. Coordinate records remain only in the replicated representation. With step_S, Legal_S, query_S, and Rel_S as defined for SidedEm-

bedRGA,

$$\begin{aligned}
\mathsf{step}(q, (t, r, \mathsf{text}(o))) &= (\mathsf{step}_S(q_T, (t, r, o)), q_D, q_M),\\
\mathsf{step}(q, (t, r, \mathsf{deleteStoreAdd}(i))) &= (q_T, q_D \cup \{i\}, q_M),\\
\mathsf{step}(q, (t, r, \mathsf{markStoreAdd}(m))) &= (q_T, q_D, q_M \cup \{m\}),\\
\mathsf{Legal}(\ell) &\iff \mathsf{Legal}_S(\mathsf{projectText}(\ell)),\\
\mathsf{Rel}((s_T, s_D, s_M), q) &\iff \mathsf{Rel}_S(s_T, q_T) \wedge s_D = q_D \wedge s_M = q_M.
\end{aligned}$$

Here $\mathsf{text}(o)$ denotes either text insertion or text deletion; $\mathsf{projectText}$ retains those events and unwraps their payloads. Store additions are idempotent. A raw text deletion has the sided step above, but issuance forbids it. The core query is

$$\mathsf{query}_{\mathrm{seq}}(q, k) = \begin{cases} \mathsf{query}_S(q_T, \mathsf{rd}), & k = \mathsf{text},\\ q_D, & k = \mathsf{deletions},\\ q_M, & k = \mathsf{marks}, \end{cases}$$

with results injected into the corresponding sum variant. In particular, logical deletion does not remove a record from this core text query. The rich query filters logically deleted characters and resolves formatting in document order; it is not the core component query. For a requested formatting kind, the rich query returns (*codepoint*, *formatted*) for each live character. The flag is true exactly when the greatest-identifier covering mark of that kind is an add; absent marks and winning remove marks give false. Coverage is a half-open interval of live positions. A dead boundary resolves to the nearest surviving character on its gravity side: right for before, left for after. An after boundary lies after that character and the following run of identifiers newer than the mark. A before start lies immediately before its resolved character; a before end moves left over the preceding newer-than-mark run. If no survivor exists, an after start uses document start and a before end uses document end; a before start or after end instead collapses the interval. Formally, for the live list $L = \mathsf{filter}(\lambda z.\ z.\mathsf{id} \neq 0 \wedge z.\mathsf{id} \notin q_D, q_T)$ and requested kind k, let w_j be the greatest-identifier mark in q_M of kind k covering position j by the boundary rules above (none if no such mark exists). Then

$$\mathsf{query}^{\mathrm{rich}}_{\mathrm{seq}}(q, k) = [(\mathsf{cp}(L_j.\mathsf{id}),\ \exists m.\ w_j = \mathsf{some}(m) \wedge m.\mathsf{operation} = \mathsf{add}) \mid 0 \leq j < |L|].$$

Here $\mathsf{cp}(i)$ is the payload at the first occurrence of i in the root-free text list; certified lists have unique identifiers. The Boolean is false for no winner or a winning remove. Issuance makes mark identifiers unique; on arbitrary raw inputs, equal-identifier ties keep the first mark in the finite-set enumeration used by the renderer. This query exposes a Boolean formatting flag, not the stored mark value.

Lean: `SidedPeritext.coreGuard`, `SidedPeritext.core_join_at`, `SidedPeritext.coreOrder`, `SidedPeritext.clientSpec`, `SidedPeritext.verified`, `SidedPeritext.richOrder`, `SidedPeritext.richVerified`.

Lean: `PeritextRender.startIncl`, `PeritextRender.endExcl`, `PeritextRender.renderMarksDoc`.

Datatype-state collection. The sided sequence and the ordinary SidedPeritext core likewise use exact state. In particular, the core query exposes its component stores, so a collector that deletes entries cannot preserve that query by equality. A representation-changing theorem must either retain those stores, change the public query boundary, or relate them through an explicit abstraction. The released rich-query package takes the latter route.

The rich package has an operational collector. A compact state contains the retained sided text, a Fugue gap summary, deletion and mark stores, and a stable Lamport cut. Its representation evidence records a keep predicate, knowledge used to reconstruct the full semantic text, gap-map correctness, an old-identifier bound, and equality of every rich-text query. The physical system can silently collect text, trim deletion metadata, drop a cancelling mark pair, or reframe identifiers; ordinary and virtual-base operations remain visible. `StateGCProtocol.refines` then covers arbitrary finite interleavings of those steps. Text collection requires a correct gap map, unique birth identifiers, deletion of every dropped character, and retention of all mark endpoints; the plan bounds dropped identifiers by its stable cut. Trimming deletion metadata removes identifiers absent from the retained text. Dropping a mark pair requires an earlier add and later remove of the same kind, endpoints, and endpoint sides, unique mark identifiers, every other same-kind mark later than the remove, and no character birth in the interval between the pair's identifiers. These are conditions of the preservation results, not unconditional permissions to discard metadata.

Lean: `SidedPeritext.StateGC.CompactState, SidedPeritext.StateGC.collectText, SidedPeritext.StateGC.trimDeleted, SidedPeritext.StateGC.dropMarkPair`.

Lean: `SidedPeritext.StateGC.Protocol.Represents, SidedPeritext.StateGC.Protocol.protocol`.

Queue. *Datatype D.* The operations are $\mathsf{enq}(v)$ and $\mathsf{deq}(i)$, where $v \in \mathbb{N}$ is the value to enqueue and $i \in \mathbb{N}$ identifies the element to dequeue. Enqueue event $(t, r, \mathsf{enq}(v))$ assigns tag t to its value. The representation is a list of tagged values, initially empty. Concrete enqueue appends (t, v) unless t is already present; concrete dequeue filters the named tag wherever it occurs. Its query returns the tagged head, or none when empty. Writing $\mathsf{tags}(X)$ for the identifiers in list X, the raw survivor list is

$$\mathsf{filter}_{i \in \mathsf{tags}(A) \cap \mathsf{tags}(B)}(L) +\!\!+ \mathsf{filter}_{i \notin \mathsf{tags}(L)}(A) +\!\!+ \mathsf{filter}_{i \notin \mathsf{tags}(L)}(B).$$

Merge sorts this list by enqueue key, so concurrent enqueue order does not depend on branch argument position.

Replay-order relation ord. For $q = (t, r, \mathsf{enq}(v))$, put $k(q) = (t, v)$. The value is a tie-breaker on raw inputs; distinct issued enqueues are ordered by timestamp. The directed policy is

$$\begin{aligned} q_1 \xrightarrow{\mathsf{ord}} q_2 &\iff k(q_1) <_{\mathrm{lex}} k(q_2) \quad \text{for two enqueues,} \\ (t, r, \mathsf{enq}(v)) \xrightarrow{\mathsf{ord}} (u, r', \mathsf{deq}(i)) &\iff t = i, \end{aligned}$$

and it is false on all other directed forms.

Issuance. Dequeue issuance names the tag at the issuer's observed head:

$$\begin{aligned} \mathsf{qApplicable}((t, r, \mathsf{enq}(v)), s) &\iff t \notin \mathsf{tags}(s), \\ \mathsf{qApplicable}((t, r, \mathsf{deq}(i)), s) &\iff \exists v, s'.\ s = (i, v) :: s'. \end{aligned}$$

Sequential specification. The state is $\Sigma_{\mathrm{seq}} = \mathsf{List}(\mathbb{N} \times \mathbb{N})$, with $q_0 = []$. Enqueue appends at the tail; dequeue removes only the named head:

$$\begin{aligned} \mathsf{step}(s, (t, r, \mathsf{enq}(v))) &= s +\!\!+ [(t, v)], \\ \mathsf{step}((i, v) :: s, (u, r, \mathsf{deq}(i))) &= s, \\ \mathsf{step}(s, (u, r, \mathsf{deq}(i))) &= s \qquad \text{if the head does not have tag } i. \end{aligned}$$

For a prefix u, let $B(u)$ be the set of timestamps of its enqueue events and $q_u = \mathsf{run}(u)$. Then $\mathsf{Legal}(\ell)$ holds exactly when every prefix decomposition satisfies

$$\begin{cases} t \notin B(u), & e = (t, r, \mathsf{enq}(v)), \\ i \in B(u) \wedge \big((\exists v, w.\ q_u = (i, v) :: w) \vee i \notin \mathsf{tags}(q_u)\big), & e = (t, r, \mathsf{deq}(i)). \end{cases}$$

In particular, removing a different live element is illegal, even though the total step leaves the queue unchanged. Previously enqueued identities are checked from the history, not retained as tombstones in abstract state. The remaining components are

$$\mathsf{Rel}(s, q) \iff s = q, \qquad \mathsf{query}_{\mathrm{seq}}(q, \mathsf{rd}) = \begin{cases} \mathsf{none}, & q = [], \\ \mathsf{some}(i, v), & q = (i, v) :: w. \end{cases}$$

The public certificate constructs a legal, loOn-respecting replay for every certified version. It groups each removed enqueue with its dequeues, then appends the surviving enqueues in their implementation-witness order. Head issuance proves that a survivor cannot causally precede an enqueue later removed as head, which justifies this rearrangement. The combined theorem `queue_correct` proves, for the same ordinary or virtual certified execution, public sequential correctness, timestamp-ordered surviving contents, and visibility-incomparability of dequeues naming the same target. For linear mint histories, `client_linear_fifo` proves equality with the ordinary FIFO value sequence. Queue is a released production package.

Lean: `Queue.qEventOrder, Queue.replayOrder, Queue.q_join_at`.

Lean: `Queue.qApplicable, Queue.queue_replay_witness, Queue.replayAdequate`.

Lean: `Queue.clientStep, Queue.clientLegalAux, Queue.clientSpec, Queue.client_linear_fifo`.

Lean: `Queue.queue_legal_witness, Queue.verified`.

Lean: `Queue.queue_correct`.

Datatype-state collection. Queue carries an exact-state baseline on its list of surviving tagged values. This records coverage, not a representation-changing reclamation theorem.

FugueMax. *Datatype D.* An operation payload contains a command $\mathsf{ins}(p, d)$ or $\mathsf{del}(i)$, a left-origin identifier, an optional right-origin identifier, and an immutable tagged chain. Here $p, i \in \mathbb{N}$ and $d \in \{\mathsf{L}, \mathsf{R}\}$. The event envelope supplies timestamp and replica; the positional issuer below constructs the remaining fields. Insertion identifiers and values both equal the event timestamp. The implementation stores a live coordinate list and a finite set of immutable insertion records, including births of deleted elements. These records retain the origins and tagged chains needed by the positional issuer; deletion-event history is not stored. Merge combines the live lists by the three-way sequence merge and unions the birth sets. The initial state is $([], \varnothing)$. Insertion adds its immutable birth record and inserts the corresponding coordinate record into the live list; deletion filters its target and preserves birth records. The query projects live identifiers.

Replay-order relation ord. The relation is false except for insertion before deletion of that identity:

$$\mathsf{ins}^{t}_{p,s} \xrightarrow{\mathsf{ord}} \mathsf{del}_i \iff t = i.$$

Issuance. Issuance accepts exactly a prepared operation produced by the positional generator on the stored live list and birth metadata. It requires positive timestamps and, for insertion, a

timestamp greater than every known birth. The existing total-index convention sends an out-of-range insertion to root and makes an out-of-range deletion a no-op targeting zero. Framework execution supplies global timestamp freshness. More explicitly, insertion at position $i > 0$ chooses the preceding live identifier as left origin a; position zero chooses root. Let n be the next birth after a in immutable display order, including deleted births. If n exists and a already has a right child, the prepared operation inserts as a left child of n; otherwise it inserts as a right child of a. Its right origin is n when present and none otherwise. The immutable chain extends the chosen parent's chain by a left entry with timestamp difference $t - n$, or a right entry with difference $t - a$ tagged by n's key (the tag is [0] when no successor exists). These are the payload fields checked by issuance, not arbitrary metadata supplied by the client. Deletion at position i names that live identifier and carries zero/none origins and an empty chain.

Sequential specification. The state is $\Sigma_{\text{seq}} = \mathsf{List}(\mathbb{N} \times \mathbb{N})$, with $q_0 = [(0, 0)]$. For the first occurrence $q = u{+}{+}[(p, y)]{+}{+}v$ of the parent,

$$\begin{aligned}
\mathsf{step}(q, e) &= u{+}{+}[(t, t), (p, y)]{+}{+}v && e.\mathsf{command} = \mathsf{ins}(p, \mathsf{L}),\\
\mathsf{step}(q, e) &= u{+}{+}[(p, y), (t, t)]{+}{+}v && e.\mathsf{command} = \mathsf{ins}(p, \mathsf{R}),\\
\mathsf{step}(q, e) &= \begin{cases} q, & i = 0,\\ \mathsf{filter}(\lambda z.\ z.\mathsf{id} \neq i, q), & i \neq 0 \end{cases} && e.\mathsf{command} = \mathsf{del}(i).
\end{aligned}$$

Here $t = \mathsf{time}(e)$; insertion returns q when its parent is absent. Origins and chains do not enter this step. Let $B(u)$ be the identifiers allocated by insertions in prefix u. Then $\mathsf{Legal}(\ell)$ requires $\mathsf{Nodup}(\mathsf{times}(\ell))$, positive timestamps, and for every prefix decomposition,

$$\begin{cases} p = 0 \vee \big(p \in B(u) \wedge \forall d \in u.\ d.\mathsf{command} \neq \mathsf{del}(p)\big), & e.\mathsf{command} = \mathsf{ins}(p, d),\\ i = 0 \vee i \in B(u), & e.\mathsf{command} = \mathsf{del}(i). \end{cases}$$

Thus a nonroot insertion parent must not yet have been deleted, but repeated deletion remains legal and idempotent. The representation relation is

$$\mathsf{listRel}(s, q) \iff \mathsf{map}(\mathsf{sProj}, s.\mathsf{live}) = \mathsf{filter}(\lambda p.\ p.\mathsf{id} \neq 0, q).$$

Its query is

$$\mathsf{query}_{\text{seq}}(q, \mathsf{rd}) = \mathsf{map}(\mathsf{id}, \mathsf{filter}(\lambda z.\ z.\mathsf{id} \neq 0, q)).$$

Each insertion's value is its own timestamp. No birth metadata occurs in this abstract state. The proof constructs parents before children and farther siblings before nearer siblings, then applies deletions. This is a witness-construction order, not another public policy. `FugueMax.correct` gives one witness satisfying this public contract and all three maximal-non-interleaving clauses for the same ordinary or virtual certified execution; it requires no separate `MaxReach` premise. Explicitly, let B contain all insertion identifiers, including deleted ones, $V \subseteq B$ the live identifiers, $\ell(x)$ and $r(x)$ the recorded left and optional right origins, and $x \prec y$ the immutable coordinate display order, also including deleted elements. Two live identifiers are consecutive when $x \prec y$ and no $z \in V$ lies strictly between them. The three clauses, for $x, y \in V$, are:

1. If $\ell(y) = x$ and $y \prec d$ for every other $d \in B$ with $\ell(d) = x$, then x, y are consecutive.
2. If $r(x) = \mathsf{some}(y)$ and $d \prec x$ for every other $d \in B$ with $r(d) = \mathsf{some}(y)$, then x, y are consecutive unless
$$\ell(x) \neq \ell(y) \ \wedge\ \exists c \in B.\ \ell(x) \prec c \prec y \wedge \neg\exists n \in \mathbb{N}.\ \ell^n(c) = \ell(x).$$
3. If $\ell(x) = \ell(y)$, $r(x) = r(y)$, and $x < y$ as identifiers, then $x \prec y$.

Competitors and the exception witness range over all births, not merely live characters; only the adjacency conclusion ignores deleted characters. The registered package is `FugueMax.verified`. No independent FugueMax runtime is claimed.

Lean: `SidedEmbedRGA.FugueMax.datatype`, `SidedEmbedRGA.FugueMax.generation`, `SidedEmbedRGA.FugueMax.replayOrder`.

Lean: `SidedEmbedRGA.FugueMax.join_of_mint`.

Lean: `SidedEmbedRGA.FugueMax.listLegal`, `SidedEmbedRGA.FugueMax.listSpec`, `SidedEmbedRGA.FugueMax.listRel`, `SidedEmbedRGA.FugueMax.verified`, `SidedEmbedRGA.FugueMax.correct`.

Lean: `SidedEmbedRGA.mGenInsAfter`, `SidedEmbedRGA.ForwardNIM`, `SidedEmbedRGA.BackwardNIExcM`, `SidedEmbedRGA.SameOriginLowFirstM`.

Datatype-state collection. Datatype-state collection is staged. The implementation retains immutable birth/origin records after deletion because later positional mints may depend on them. The public plain-list abstraction does not establish that this metadata can be discarded. No collector is included in the registered package.

11.2 Proof routes and evidence status

Grow-only stores and counters share a proof route: their commuting updates satisfy universal `MergeLaws`, `DeltaLaws`, and `CommutingPeelLaw`, yielding all-context Join and total sequential correctness for their respective contracts.

Released family	Merge-proof route
Grow-only stores, flat counters, bounded counter, LWW register, TreeMove, AegisSheet, RGA	Universal equations adapted to `CanonicalJoinLaws`, then all-context `Join`
Efficient OR-set	Direct execution invariant and exact-state replay, including recursive virtual merge bases; not arbitrary-state Join
Queue	Context-local `JoinAt` under issuance honesty, followed by legal FIFO replay and the concurrent-dequeue contract
MVR	Surviving-write execution invariant, including recursive virtual merge bases, followed by the causally maximal writes contract under issuance
EmbedRGA and SidedEmbedRGA	Direct datatype-specific `JoinAt` under `EHonestCore` or `SHonestCore`
FugueMax	Direct `JoinAt` from exact framework issuance; the public witness also satisfies maximal non-interleaving
Peritext	Inherited payload-parametric EmbedRGA certificate
SidedPeritext	Componentwise `joinAt_prod`: restricted sided sequence plus two all-context grow-only stores

The negative and partial-evidence ledger is separate from the production registry. The multi-value register's counterexample rules out the ordinary single-register specification for concurrent executions; its appropriate multi-value public contract is now certified. The old coordinate-only

FugueMax model remains a replay-only companion; the enriched implementation above has its own public certificate. Negative and replay-only declarations do not substitute for public datatype packages.

Lean: `Instances.MVR.concurrentState_no_sequential_register`.

Lean: `Queue.replayAdequate`.

Lean: `SidedEmbedRGA.fuguemax_replay_witness`.

Lean: `SidedEmbedRGA.fuguemax_backward_ni`.

A Exact correspondence index

The companion Lean ledger imports these modules and checks the named declarations, so a rename or type-level disappearance fails the document gate.

Concept	Lean declaration	Source
MRDT interface	`MRDTSig`	`Signature.lean`
Events and internal replay support	`Op`, `UpdateSig`	`UpdateSignature.lean`
Replay lists	`applySeq`, `listPermOf`, `respects`	`Replay.lean`
Operational semantics	`Configuration`, `Label`, `Step`, `StepV`	`Execution.lean`
Store/GCA events	`StoreInv`, `gca_events_of_storeInv`	`StoreInvariant.lean`
Event replay order	`EventOrder`, `UpdateSig.replayBefore`	`UpdateSignature.lean`
Set-relative linearization	`loOn`, `IsCanonicalState`	`SetRelativeReplay.lean`
Replay laws	`ReplayLaws`	`ReplayLaws.lean`
Merge semantics	`JoinAt`, `Join`, `JoinOn`	`MergeLaws.lean`
Canonical-state Join laws	`CanonicalJoinLaws`, `CanonicalJoinLaws.join`; universal adapter `CanonicalJoinLaws.ofArbitrary`	`MergeLaws.lean`; `Adequacy.lean`
Canonical reachability	`CanonicalConfig`, `replayWitness_of_join`	`Adequacy.lean`
Issuance/spec interfaces	`Issuance`, `SequentialSpec`	`Certificates.lean`
LWW single-*ord* ordering	`LWWRegister.replayOrder_order_acyclic`, `LWWRegister.canonical_respects`, `LWWRegister.verified`	`LWWRegister.lean`
Certified adequacy	`replayWitness_of_mintCertified`	`CertifiedAdequacy.lean`
Client theorem	`IsSpecLinearizable`, `VerifiedMRDT`	`Correctness.lean`
Virtual construction	`virtualMergeBaseState`	`VirtualMergeBase.lean`
Virtual adequacy	`virtualMergeBaseState_canonical`, `replayWitnessV_of_join`	`VirtualAdequacy.lean`
History GC	`GC.Certificate`, `GC.execution_refines_noGC`	`Distributed.lean`
Compressed DAG	`GC.CompressedReaches`	`CompressedDAG.lean`
Runtime refinement	`GC.runtime_refines_coreV`	`Refinement.lean`
Local state-GC certificate	`StateGCCertificate`	`StateGC.lean`
Datatype-state GC protocol	`StateGCProtocol`	`StateGC.lean`
Per-datatype GC coverage	`Production.StateGC.registry`	`StateGCCoverage.lean`
GC composition	`GC.CombinedSteps.refinesV`	`StateComposition.lean`

B Historical binary boundary and negative evidence

The merge-free projection D_U from Section 2 is implemented by `UpdateSig`. The projection `MRDTSig.toUpdateSig` forgets queries and ternary merge, retaining only the data needed for event application, finite replay, and commutation. It is the internal algebraic view consumed by replay proofs.

Some historical proof modules still study a binary operation. They request it through a separate optional capability:

$$\mathsf{HistoricalBinaryMerge}(D_U) = (\mathsf{merge}_2 : \Sigma \to \Sigma \to \Sigma).$$

Lean exposes the operation as `UpdateSig.historicalMerge`, keeping it visibly distinct from the ternary `MRDTSig.merge`. For an MRDT D, the compatibility instance `MRDTSig.historicalBinaryMerge` supplies the initial-state slice

$$\mathsf{merge}_2(a, b) = D.\mathsf{merge}(\sigma_0, a, b).$$

Binary merge is therefore neither a field of `UpdateSig` nor part of the live execution or `VerifiedMRDT` interfaces. It appears only when a historical theorem explicitly requests it.

Lean: `Foundation.UpdateSig, Foundation.HistoricalBinaryMerge, Foundation.UpdateSig.historicalMerge`.

Lean: `MRDTSig.toUpdateSig, MRDTSig.historicalBinaryMerge`.

Historical binary VC bundles and countermodels are not premises of the public correctness theorem. Two negative results are retained as audit controls:

1. convergence over a merely backward-closed version set can fail when the absorber search ranges over a later whole configuration; and
2. associativity, commutativity, and idempotence of binary merge do not by themselves imply the historical `BinaryJoin` property. This countermodel does not by itself state a negative theorem about ternary Join.

They are machine-checked in negative-evidence modules and remain outside the production package.

Lean: `Foundation.convergence_over_backward_closed_subsets_false`.

Lean: `Foundation.binaryLaws_insufficient`.

C Notation and elaboration notes

- The paper writes S, H, and L as partial maps. Lean totalizes them with `Option`: $S(v) = (s, E)$ abbreviates `C.ver v = some (s,E)`, and $H(r) = v$ abbreviates `C.head r = some v`.
- $f(x) = \bot$ means $x \notin \mathsf{dom}(f)$; for a totalized Lean map, this is equality with `none`.
- Lean sets have type `Set alpha`. Finiteness is normally witnessed by a list satisfying `listPermOf`, rather than stored in the set type.
- $v < v'$ in allocation rules is the natural-number rank of version identifiers. Event Lamport time is the first component of an event and is constrained separately.
- The companion ledger uses the actual Lean namespaces and checks elaboration of every principal declaration cited here.

D Merge-law adapters

This appendix expands the sufficient proof route from Section 6. It is independent of the client contract: a datatype may instead establish replay adequacy directly from an execution invariant.

The universal equations construct the canonical contract:

$$\begin{aligned} \mathsf{MergeLaws}(D) &\Longrightarrow \mathsf{JoinCoreLaws}(D), \\ \mathsf{MergeLaws}(D) \wedge \mathsf{DeltaLaws}(D) &\Longrightarrow \mathsf{FeasibleDeltaLaws}(D), \\ \mathsf{MergeLaws}(D) \wedge \mathsf{DeltaLaws}(D) \wedge \mathsf{CausalDeltaLaw}(D) &\Longrightarrow \mathsf{CanonicalJoinLaws}(D). \end{aligned}$$

Lean: `MergeLaws.toJoinCore`.

Lean: `feasibleDeltaLaws_of_delta, CanonicalJoinLaws.ofArbitrary`.

The bridge uses `MergeLaws` for the common core and canonical unit law, substitutes $\mathsf{do}(m, e)$ for the arbitrary delta-state argument in each `DeltaLaws` equation, and discards the canonical-tuple premises.

The adapter `JoinProof.ofArbitraryStateLaws` first constructs the canonical law bundle and invokes its Join theorem. Both proof routes therefore conclude the same `Join` property.

Lean: `JoinProof.ofArbitraryStateLaws`.

RGA instance. RGA takes the universal shortcut. Its pointwise Boolean merge proves `MergeLaws` and `DeltaLaws`; raw updates commute, so `CommutingPeelLaw` supplies `CausalDeltaLaw`. The resulting chain is

$$\mathsf{MergeLaws} + \mathsf{DeltaLaws} + \mathsf{CommutingPeelLaw} + \mathsf{allComm} \Longrightarrow \mathsf{CanonicalJoinLaws} \Longrightarrow \mathsf{Join}(\mathsf{RGAM}).$$

The issuer predicate `RGA.applicable` is used later to construct a legal ordinary-list history; the RGA merge theorem itself holds for every replay context.

Lean: `RGA.RGAM_mergeLaws, RGA.RGAM_deltaLaws, RGA.join`.